

Transformer-based multi-agent reinforcement learning for flexible job shop scheduling with AGVs

Yuxiang Shen , Xutang Zhang*, Tianguo Jin*

School of Mechatronics Engineering, Harbin Institute of Technology, 92 West Dazhi Street, Harbin, 150001, Heilongjiang, China

HIGHLIGHTS

- A cooperative dual-agent Transformer framework solves FJSP with AGV constraints.
- Global self-attention mechanism effectively balances AGV loads and reduces idle time.
- MA-Trans outperforms state-of-the-art graph-based and recurrent DRL baselines.
- Parallelizable architecture reduces training time by 23.8% with millisecond-level inference.
- Achieves robust zero-shot generalization across variable problem scales without retraining.

ARTICLE INFO

Keywords:

Transformer
Deep reinforcement learning
Automated guided vehicle (AGV)
Multi-agent reinforcement learning
Flexible job shop scheduling problem

ABSTRACT

Integrated scheduling in flexible job shops with AGV transportation constraints presents a complex optimization challenge due to the tight coupling between operation sequencing and material handling. Existing reinforcement learning methods, typically relying on recurrent or fully connected architectures, often struggle to capture global spatio-temporal dependencies and suffer from sequential computational bottlenecks. To address these limitations, this study proposes MA-Trans, a hierarchical deep reinforcement learning framework based on Proximal Policy Optimization and Transformer encoders. By designing a cooperative dual-agent architecture, the model jointly optimizes job scheduling and AGV dispatching, employing self-attention mechanisms to efficiently encode long-range interactions among manufacturing resources. Experimental results demonstrate that MA-Trans significantly outperforms state-of-the-art baselines, including ResGAT and LSTM-Ptr, in both makespan minimization and AGV load balancing. Beyond solution quality, the proposed Transformer-based architecture reduces training time by approximately 23.8% compared to recurrent baselines while maintaining low-latency inference of around 2.75 ms. Moreover, the model exhibits robust zero-shot generalization capabilities, adapting to varying problem scales and AGV configurations without retraining, thereby highlighting its potential for real-time smart manufacturing applications.

1. Introduction

With the acceleration of digital transformation in manufacturing, Intelligent Flexible Manufacturing Systems (FMS) have become pivotal for high-mix, low-volume production. In such systems, the seamless coordination between job scheduling and AGV-based material handling is a critical determinant of overall efficiency [19]. As a key logistical resource, Automatic Guided Vehicles (AGVs) dictate the production rhythm; however, their integration introduces complex coupling constraints. In the absence of a unified scheduling framework, the misalignment between processing and transportation tasks frequently

leads to machine idling and path conflicts, ultimately degrading system throughput [23,36].

The Flexible Job Shop Scheduling Problem (FJSP) with AGV constraints is a well-known NP-hard combinatorial optimization problem [6]. While traditional approaches such as Mixed-Integer Linear Programming (MILP) and Genetic Algorithms (GA) have been extensively applied, they often suffer from limited scalability and responsiveness when facing dynamic shop-floor disturbances [27,32]. Consequently, the manufacturing research community is increasingly shifting toward data-driven intelligence. This paradigm has already proven highly effective in process-level modeling and control. For

* Corresponding author.

Email addresses: 23s136348@stu.hit.edu.cn (Y. Shen), zxt@hit.edu.cn (X. Zhang), jintg@hit.edu.cn (T. Jin).

example, recent studies have successfully utilized hybrid deep learning models to predict weld pool dimensions and optimize laser welding parameters with high precision [1,4,10]. While these studies verify the power of data-driven methods in optimizing local physical processes, applying such intelligence to the system-level coordination of heterogeneous resources (jobs, machines, AGVs) presents a different class of challenges characterized by high-dimensional state spaces and intricate logical coupling.

In recent years, Deep Reinforcement Learning (DRL) has shown promise in addressing these dynamic scheduling complexities [8]. However, most existing DRL-based scheduling approaches rely on multilayer perceptrons (MLPs) or Convolutional Neural Networks (CNNs) with fixed input dimensions. These architectures suffer from a fundamental structural limitation: they struggle to capture the long-range dependencies and dynamic topology among heterogeneous resources (jobs, machines, and AGVs). Crucially, the fixed network structure constrains the expressive power of learned policies, often resulting in poor zero-shot generalization when the production scale changes [13]. This limitation necessitates a more flexible architecture capable of processing variable-length state sequences and modeling complex relational couplings.

To bridge the gap between structural rigidity and system adaptability, this study proposes a Multi-Agent Transformer-based Reinforcement Learning (MA-Trans) framework [28] for multi-resource coordination. Unlike traditional flat architectures, we construct a dual-agent cooperative mechanism that explicitly decouples the complex joint scheduling problem into operation sequencing and transportation dispatching. The Job Agent utilizes a Transformer encoder to process variable-length state sequences, capturing global dependencies among jobs to support zero-shot generalization across different problem scales. Simultaneously, the AGV Agent dynamically allocates transportation resources based on real-time system states. To further enhance computational efficiency and training stability, machine selection is streamlined through a knowledge-guided heuristic mechanism that evaluates candidate machines by optimizing combined processing and transportation costs. This hybrid paradigm effectively resolves the scalability issues inherent in MLP-based methods while maintaining high solution quality.

The main contributions of this study are summarized as follows:

1. A unified state representation scheme is developed to encode the heterogeneous states of jobs, machines, and AGVs into standardized token sequences. This design overcomes the fixed-input limitation of traditional architectures, enabling the policy to process variable-scale production instances without retraining.
2. A Multi-Agent Transformer-based RL (MA-Trans) framework is proposed, which decouples the joint scheduling task into operation sequencing and transportation dispatching. This structure reduces the action space complexity from exponential $\mathcal{O}(|J| \times |V|)$ to linear $\mathcal{O}(|J| + |V|)$, thereby addressing the curse of dimensionality and improving training stability.
3. A hybrid decision mechanism is integrated, combining the learned policy with knowledge-guided heuristics for machine selection. Extensive experiments demonstrate that the proposed method achieves faster convergence and robust zero-shot generalization across unseen task scales compared to state-of-the-art baselines.

2. Related work

In recent years, research on flexible job shop scheduling (FJSP) has advanced significantly within the context of manufacturing system optimization, particularly under scenarios involving the coupling of processing equipment and AGV-based transportation resources, which substantially increases the complexity of scheduling decisions. To provide a clearer understanding of the current research landscape, the related literature is broadly categorized into three areas: first, a review

of traditional optimization approaches and their limitations in job shop scheduling; second, a summary of recent developments in reinforcement learning-based scheduling models; and third, an overview of preliminary attempts to incorporate Transformer architectures into scheduling optimization. This structured analysis serves to contextualize the present study and highlight its position and contributions within the existing body of research.

2.1. Non-learning-based method

In job shop environments, the coordinated scheduling of processing machines and AGVs refers to the simultaneous allocation of machining equipment and transportation resources to pending operations under given production conditions, with the objective of jointly optimizing processing tasks and material handling. Existing studies predominantly adopt mathematical modeling approaches, such as Mixed-Integer Programming (MIP), Petri nets, and disjunctive graphs, in combination with metaheuristic optimization algorithms to derive scheduling solutions and enable multi-resource collaborative control [26].

Early studies employed exact methods to ensure optimality. For instance, Fontes et al. [11] developed a Mixed-Integer Programming (MIP) model, while Baruwa et al. [5] utilized time-colored Petri nets (TCPN). However, these approaches suffer from combinatorial explosion, making them computationally prohibitive for large-scale instances. To address larger-scale complexities and multi-objective requirements, evolutionary algorithms have been widely adopted. For instance, Shi et al. [24] proposed a hybrid NSGA-II to solve the energy-efficient flexible job shop scheduling problem with finite multi-load AGV resources. However, to further enhance the balance between global exploration and local exploitation, hybrid scheduling approaches have gained prominence. Xie et al. [33] proposed a hybrid genetic algorithm combined with a branch-and-bound strategy to address joint scheduling in multi-shop environments. Building on this hybrid paradigm, Memetic Algorithms (MA), which explicitly integrate global evolutionary search with problem-specific local search mechanisms, have become a mainstream baseline for solving complex joint scheduling problems. For instance, Wang et al. [29] proposed a two-phase cascaded memetic algorithm to effectively optimize the integrated scheduling problem considering AGV transportation constraints.

Despite their effectiveness, these non-learning methods share a fundamental limitation: they are static. Any unexpected disturbance requires a complete rescheduling process, which is computationally expensive. Furthermore, they lack generalization capability; a solver optimized for one scenario cannot be directly applied to another without re-optimization. These limitations motivate the shift toward Reinforcement Learning (RL), which offers the potential for adaptive, zero-shot generalization across varying problem scales.

2.2. RL-based method

Reinforcement Learning (RL), as an autonomous decision-making framework based on agent–environment interaction, has gained increasing attention in job shop scheduling problems in recent years. Compared to traditional optimization approaches, RL does not rely on explicitly defined mathematical models; instead, it learns scheduling policies through trial-and-error mechanisms within high-dimensional state spaces. This enables greater adaptability and generalization capability, making it well-suited for flexible manufacturing systems characterized by complex resource relationships and diverse scheduling constraints. In the broader manufacturing context, data-driven intelligence has already proven effective for process-level optimization, such as using artificial neural networks for weld quality prediction [2] and machine learning-assisted evolutionary algorithms for performance evaluation [14]. Aligning with this trend, the application of RL in intelligent manufacturing scheduling continues to expand, and it has been shown to offer promising generalization and flexibility in modeling complex decision-making scenarios [38].

Scheduling tasks in flexible job shops can be formulated as a Markov Decision Process (MDP), where a scheduling agent observes the shop-floor state, executes scheduling actions, and receives reward feedback from the environment, thereby learning an optimized policy through continuous interaction [17]. Leveraging adaptive policy updates and model-free learning capabilities, reinforcement learning has been widely applied to scheduling optimization tasks across various manufacturing scenarios. For instance, Luo et al. [21] employed a Double Deep Q-Network (Double DQN) to solve the FJSP, incorporating six composite dispatching rules to enable joint scheduling of jobs and AGVs. Zhang et al. [34] proposed a deep reinforcement learning approach to learn priority dispatching rules, combined with a Graph Neural Network (GNN) to extract the topological features of scheduling states. Their method demonstrated superior policy performance compared to benchmark heuristics across multiple flexible job shop test cases, validating the effectiveness of graph-based representations in handling irregular shop floor structures.

Given the strong coupling among jobs, machines, and transportation resources in joint scheduling tasks, recent studies have introduced hierarchical and distributed reinforcement learning architectures to enhance policy expressiveness and system-level coordination. Wang et al. [30] developed a hierarchical reinforcement learning model, where sub-policies were used to optimize local scheduling behaviors, resulting in improved global delivery performance. Zhou et al. [37] proposed a dual-agent DQN framework that separately models job and AGV decisions to jointly minimize makespan and balance AGV workloads. To improve scalability in large-scale manufacturing systems, Liao et al. [16] constructed cell-level agents based on Deep Recurrent Q-Networks (DRQN) with LSTM units to capture temporal dependencies in the production flow. By coordinating them via the QMIX algorithm, they achieved gains in both distributed scheduling efficiency and generalization. Beyond hierarchical structures, several works have explored more fine-grained agent decompositions for subtask-level coordination. For instance, Meng et al. [22] proposed a DMA-DDQN framework with three types of agents responsible for job assignment, operation selection, and AGV scheduling, integrating state compression and reward shaping to enhance training efficiency and generalization. Jing et al. [12] further incorporated a centralized training and decentralized execution paradigm, introducing distinct agents for jobs, machines, and scheduling controllers, thereby improving policy robustness and cooperative optimization across the system.

In summary, reinforcement learning has shown considerable potential in flexible job shop scheduling problems, offering strong self-adaptive capabilities and effective policy optimization. Particularly, the introduction of multi-agent frameworks and hierarchical modeling has significantly improved the coordination and adaptability of scheduling systems. However, traditional methods utilizing multilayer perceptrons rely on fixed input dimensions, which tightly couples the policy to specific resource scales. While recurrent neural networks and graph neural networks can handle variable inputs, they exhibit inherent drawbacks. Recurrent models often suffer from memory forgetting in long-horizon tasks and lack parallel processing capabilities, whereas graph-based methods incur high computational costs without effectively capturing long-term sequential dependencies. To address these challenges, recent studies have begun to explore network architectures capable of handling variable-length inputs and dynamically structured states. Among them, the Transformer architecture has attracted growing attention due to its self-attention mechanism and its ability to flexibly model sequential data, offering promising potential for learning expressive representations in complex scheduling environments.

2.3. Transformer-based method

To further enhance the adaptability of scheduling policies in dynamically changing resource environments, researchers have explored the integration of Transformer architectures to improve the quality of state

representation. The Transformer model is capable of uniformly processing variable-length sequences and capturing long-range dependencies among jobs, machines, and AGVs on the shop floor. This architecture has shown promising potential in several recent studies on scheduling optimization.

Originally introduced for natural language processing tasks, the Transformer architecture has demonstrated exceptional performance in sequence modeling and contextual representation through its multi-head attention mechanism. In recent years, this architecture has been increasingly adopted in combinatorial optimization problems such as job scheduling and resource allocation. Li et al. [15] proposed a Transformer-based deep reinforcement learning model that utilizes variable-length state representation matrices and a dual-head attention mechanism to jointly model job-machine states in parallel machine scheduling, significantly improving adaptability to changing problem scales. Similarly, Zhang et al. [35] developed a Scalable Graph Transformer framework designed for large-scale scheduling problems. Their approach effectively handles varying problem sizes and complex constraints, demonstrating strong generalization capabilities across different shop configurations. In parallel, researchers have approached scheduling as a sequence-to-sequence generation task. For instance, Chen et al. [7], employed an encoder-decoder architecture combined with Pointer Networks to map production states directly to scheduling sequences. While effective for planning tasks, this generative paradigm often struggles to adapt to instant dynamic disturbances and the complex real-time coupling required for AGV interactions. Taking a different perspective, Dong et al. [9], utilized a Graph Attention Network (GAT) where the shop floor topology is explicitly represented by high-dimensional adjacency matrices. Although this graph-based matrix representation captures detailed local dependencies between agents, the quadratic computational complexity associated with processing dense adjacency matrices imposes significant memory overhead, limiting the method's efficiency in large-scale instances.

In the context of joint scheduling with transportation resources, Wang et al. [31] designed a Transformer-based structure to embed job state features and facilitated action selection via the PPO algorithm, effectively solving the flexible job shop scheduling problem with AGV constraints. Furthermore, to address the complexity of heterogeneous resources, Liu et al. [18] introduced a Heterogeneous Graph Transformer (HGT) to model the dynamic interactions between jobs and machines. These studies demonstrate the architecture's capability in handling complex multi-modal data and resource coupling.

Although the aforementioned studies have preliminarily explored the potential of Transformer architectures in specific scheduling scenarios, significant challenges remain. Most existing approaches rely on flat attention mechanisms or complex graph structures, which often fail to explicitly decouple the distinct operational logic between manufacturing and transportation. Consequently, these methods lack a unified hierarchical mechanism to coordinate heterogeneous resources efficiently, leading to optimization difficulties when the system scale expands. Furthermore, there is still a lack of systematic modeling and validation of policy generalization in joint scheduling tasks involving jobs, machines, and transportation resources, leaving the potential of zero-shot transferability largely untapped.

2.4. Summary and research gaps

A synthesized comparison of the representative scheduling methodologies discussed above is presented in Table 1. As summarized in the table, traditional optimization methods (Section 2.1) ensure solution quality but suffer from static rigidity and high computational costs, making them unsuitable for real-time rescheduling. While DRL-based approaches (Section 2.2) improve responsiveness, standard MLP or CNN architectures are constrained by fixed input dimensions, which limits their zero-shot generalization across varying production scales. Furthermore, although recent Transformer-based studies (Section 2.3)

Table 1
Comprehensive comparison of representative scheduling methods and the proposed framework.

Reference	Method Class	Architecture	Action Complexity	Task Scope	Zero-Shot Gen.	Key Limitation / Advantage
Optimization & Meta-heuristics (Section 2.1)						
Fontes et al. [11]	MIP	Centralized	N/A (Solver)	FJSP + AGV	No	Computationally prohibitive (> 1h)
Baruwa et al. [5]	Petri Net	Centralized	N/A (Logic)	FJSP + AGV	No	State space explosion
Shi et al. [24]	Hybrid NSGA-II	Centralized	N/A (Evolution)	FJSP + AGV	No	Static; slow rescheduling
Xie et al. [33]	Hybrid GA	Centralized	N/A (Search)	Multi-Shop	No	Limited real-time responsiveness
Wang et al. [29]	Memetic Alg.	Centralized	N/A (Local Opt.)	Multi-Shop	No	Lacks zero-shot generalization
Deep Reinforcement Learning (Section 2.2)						
Luo et al. [21]	DRL	Single-Agent	Fixed Vector	FJSP + AGV	No	Fixed input; no generalization
Zhang et al. [34]	GNN-RL	Single-Agent	$\mathcal{O}(N ^2)$ (Graph)	FJSP	Yes (High Latency)	High inference cost; ignores AGV
Wang et al. [30]	HRL	Hierarchical	Fixed Vector	FJSP	No	Limited global coordination
Zhou et al. [37]	MARL	Dual-Agent	Fixed Vector	FJSP + AGV	No	Fixed dimension; weak scalability
Liao et al. [16]	MARL (LSTM)	Multi-Agent	Coupled	Cell-Shop	Yes (Seq)	Catastrophic forgetting ; serial
Meng et al. [22]	DMA-DDQN	Multi-Agent	Factorized	FJSP + AGV	No	Complex reward shaping needed
Jing et al. [12]	CTDE-RL	Multi-Agent	Centralized	FJSP + AGV	No	Non-stationarity; hard to train
Transformer-Based Methods (SOTA Competition - Section 2.3)						
Li et al. [15]	Transformer-RL	Single-Agent	$\mathcal{O}(J)$	Parallel M.	Yes	Single-task only (No AGV)
Zhang et al. [35]	Scalable GT	Single-Agent	$\mathcal{O}(N ^2)$ (Dense)	Large JSP	Yes (Costly)	Computational overhead
Wang et al. [31]	PPO-Trans.	Multi-Agent	Exp. $\mathcal{O}(J \times V)$	FJSP + AGV	Yes (Unstable)	Exploding action space
Liu et al. [18]	HGT-RL	Single-Agent	High (Hetero-Graph)	FJSP + AGV	Yes (Complex)	Slow convergence; complex mapping
Chen et al. [7]	Fed-PtrNet	Enc-Dec	$\mathcal{O}(J ^2)$	Vehicle Plan	No	Lack of real-time interactivity
Dong et al. [9]	GAT-Matrix	Multi-Agent	$\mathcal{O}(N ^2)$	MA-JSP	Partial	Heavy adjacency matrix calc.
Proposed	MA-Trans	Dual-Agent	Linear $\mathcal{O}(J + V)$	FJSP + AGV	Yes (Robust)	(Ours) Linear Scalability

Note: (1) **Notation:** $|J|$ and $|V|$ denote the number of jobs and AGVs, respectively. $\mathcal{O}(\cdot)$ indicates asymptotic complexity.

(2) **Action Complexity:** “Exp.” (Exponential) implies a coupled action space ($\mathcal{O}(|J| \times |V|)$), often leading to the curse of dimensionality and training instability. “Linear” implies a decoupled action space ($\mathcal{O}(|J| + |V|)$) via hierarchical coordination, ensuring training efficiency.

(3) **Zero-Shot Gen.:** “Yes (Stable)” indicates that the model generalizes to variable problem scales (e.g., from 10×5 to 100×20) without retraining.

Highlight: our method is the only listed framework that simultaneously achieves linear complexity and stable zero-shot generalization.

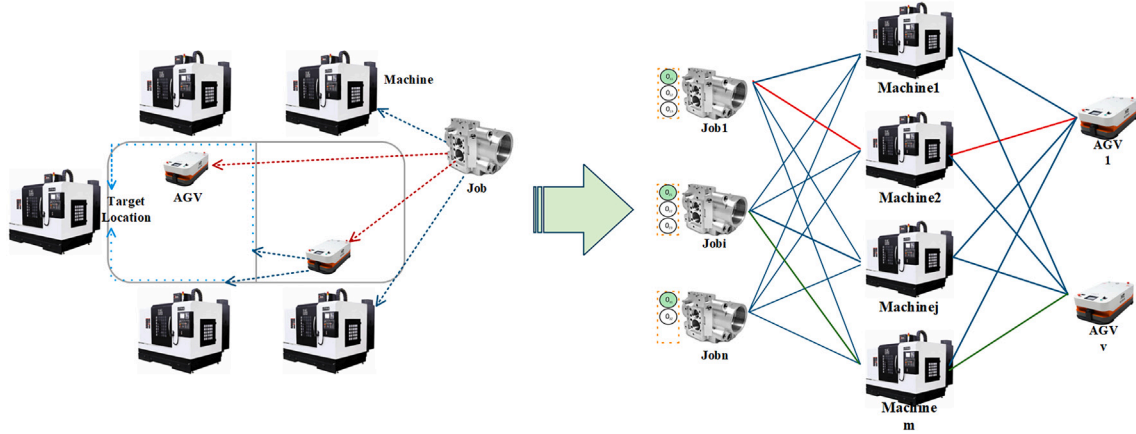


Fig. 1. Multi-Resource Collaborative Scheduling Framework for FJSP-AGV.

have addressed the variable-input challenge, most existing multi-agent frameworks still struggle with exponentially coupled action spaces ($\mathcal{O}(|J| \times |V|)$) and lack effective coordination mechanisms for heterogeneous resources.

To bridge these identified gaps, this study proposes a hierarchical Transformer-RL framework. By explicitly decoupling the joint decision process into hierarchical sub-tasks, our approach simultaneously achieves linear action complexity ($\mathcal{O}(|J| + |V|)$) and structural transferability, offering a robust solution for scalable multi-resource coordination in flexible manufacturing systems.

3. Problem description

This study investigates the Flexible Job Shop Scheduling Problem with AGV constraints, denoted as FJSP-AGV, which is an extended variant of the classical FJSP where Automated Guided Vehicles serve as essential material handling resources. Unlike the conventional FJSP that

focuses solely on the assignment of operations to machines, this problem introduces a temporal and spatial coupling between production and logistics. Specifically, whenever consecutive operations of a job are assigned to different machines, an AGV must be scheduled to perform the inter-machine transfer. Consequently, the system efficiency depends not only on machine utilization but also on the synchronous coordination of limited transportation resources. Fig. 1 illustrates the multi-resource collaborative scheduling framework modeled in this study. As depicted, the physical shop-floor entities – including jobs, machines, and AGVs – are logically mapped into a coupled network, where the completion of each job operation relies on the synergistic cooperation between specific machining resources and available transportation agents.

Theoretically, the integration of AGVs transforms the scheduling problem from a pure operation sequencing task into a dual-resource constrained optimization problem. This significantly escalates the computational complexity, rendering the problem strictly NP-hard. The core challenge lies in the dynamic interaction between subsystems: a delay

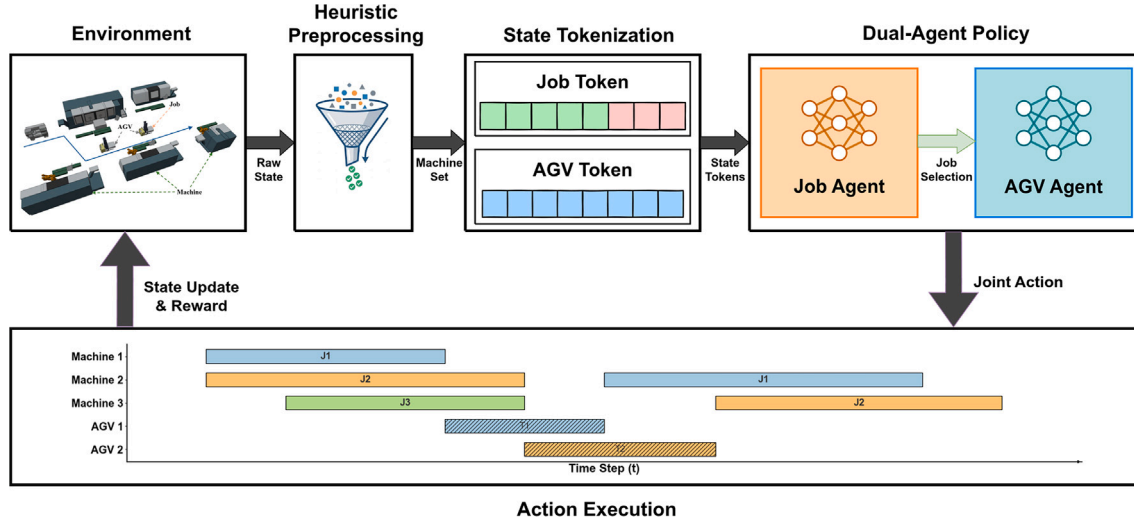


Fig. 2. Overall framework of the proposed hierarchical Transformer-based scheduling method.

in transportation can propagate through the schedule, causing machine idling and blocking, while poor machine assignment can lead to AGV congestion. Therefore, effective scheduling requires a holistic policy that jointly optimizes job selection, machine assignment, and AGV dispatching.

Specifically, this study focuses on the high-level task allocation decision layer rather than the low-level motion control layer. While path planning and collision avoidance are critical for physical implementation [39], we assume these are handled by the underlying AGV control system. Consequently, our objective is to construct a global scheduling policy, formally designated as the Multi-Agent Transformer (MA-Trans) framework, that minimizes the makespan by adaptively managing resource constraints in a high-dimensional state space. The specific assumptions and mathematical formulation are detailed as follows.

4. Methods

4.1. Proposed scheduling framework

This study adopts a hierarchical reinforcement learning approach based on the Transformer architecture to address the integrated scheduling of machines and AGVs. Inspired by the agent collaboration strategy proposed by Liu et al [20], we design a multi-agent structure that explicitly decouples job sequencing from AGV dispatching. As visually summarized in Fig. 2, the proposed framework operates as a closed-loop system consisting of four key functional modules:

1. **Environment Modeling:** The process begins with the physical shop floor (top-left), which acts as the interaction environment providing real-time *Raw State* data of jobs, machines, and AGVs.
2. **Heuristic Preprocessing:** To handle the large-scale combinatorial space, a knowledge-guided **Heuristic Preprocessing** module (top-center) filters the candidate machines. This step generates a refined *Machine Set*, effectively reducing the decision search space before neural network processing.
3. **State Tokenization:** The preprocessed states are then transformed into structured *State Tokens* via the State Tokenization module. This converts heterogeneous resource information into a standardized sequence format compatible with Transformer inputs.
4. **Dual-Agent Policy:** The core decision-making takes place in the **Dual-Agent Policy** module (top-right). This follows a hierarchical sequence: the **Job Agent** first determines the priority job

based on global dependencies, and passes this *Job Selection* decision to the **AGV Agent**, which subsequently selects the optimal transportation resource.

Finally, the resulting *Joint Action* [*Job*, *Machine*, *AGV*] is executed in the Action Execution module (bottom). This triggers a state transition in the environment and generates feedback (*State Update & Reward*) to update the policy networks, continuing iteratively until all jobs are completed.

4.2. Problem formulation & system modeling

4.2.1. Modeling assumptions and justification

To formulate the mathematical model without loss of generality and to focus on the core scheduling optimization, the following assumptions are made regarding the system environment. Justifications are provided to clarify the validity of these simplifications within the scope of upper-level task allocation:

1. **Job and Operation Constraints:** All jobs and machines are assumed to be available at the initial time ($t = 0$). Operations of the same job must be executed sequentially according to the precedence constraints, and preemption is not allowed.
Justification: This adheres to the standard definition of the flexible job shop scheduling problem. The non-preemption constraint reflects the physical requirements of precision machining, where interrupting a process often degrades surface quality.
2. **Machine Constraints:** Each machine can process at most one operation at a time. The setup time is included in the processing time. Furthermore, buffer capacities at input/output stations are assumed to be sufficient to hold waiting jobs.
Justification: The assumption of infinite buffer capacity is standard in scheduling literature to prevent system-level deadlocks caused solely by local blocking. This allows the study to evaluate the algorithm's intrinsic ability to optimize material flow and reduce work-in-process inventory naturally, rather than constraining performance via hard storage limits.
3. **AGV Capacity and Continuity:** Each AGV possesses a unit load capacity, carrying exactly one job at a time. The transportation process, comprising both empty travel and loaded travel, is considered non-interruptible. However, AGVs are permitted to wait at the machine location if the destination resource is not immediately available. Battery capacity is assumed to be sufficient for the scheduling horizon.

Table 2
Mathematical notations and decision variables.

Symbol	Description
<i>Sets and Indices</i>	
$\mathcal{J}$	Set of jobs, indexed by $i \in \{1, \dots, n\}$
$\mathcal{M}$	Set of machines, indexed by $k \in \{1, \dots, m\}$
$\mathcal{V}$	Set of AGVs, indexed by $v \in \{1, \dots, w\}$
$O_{i,j}$	The j -th operation of job J_i , where $j \in \{1, \dots, n_i\}$
<i>Parameters</i>	
n_i	Total number of operations for job J_i
$p_{i,j,k}$	Processing time of operation $O_{i,j}$ on machine M_k
$t_{u,v}$	Transportation time between machine M_u and M_v
L	A sufficiently large positive constant
<i>Decision Variables</i>	
$x_{i,j,k}$	Binary variable; 1 if $O_{i,j}$ is processed on M_k , 0 otherwise
$y_{i,j,v}$	Binary variable; 1 if the transfer of $O_{i,j}$ is assigned to AGV V_v
$S_{i,j}$	Start time of operation $O_{i,j}$
$C_{i,j}$	Completion time of operation $O_{i,j}$
W_v	Total number of tasks assigned to AGV v
$\bar{W}$	Average workload of all AGVs

Justification: The unit load configuration reflects the standard setup of AGVs in flexible workshops. Neglecting battery charging allows the model to focus on the efficiency of task dispatching logic without the interference of energy management constraints.

4. **AGV Routing and Traffic:** AGVs are assumed to travel along the shortest paths between nodes. While travel times are calculated based on distance, dynamic collision avoidance and deadlocks are not explicitly modeled in this upper-level optimization.

Justification: This study focuses on the joint scheduling of resources rather than path planning. It is assumed that a hierarchical control architecture is in place, where a lower-level traffic control system handles real-time collision avoidance and local path re-planning. Consequently, the transportation durations utilized in this model are treated as expected values that include a margin for potential traffic delays, ensuring the schedule remains robust.

4.2.2. Scheduling constraints & objective

Based on the assumptions outlined in Section 4.2.1, the FJSP-AGV problem is formulated as a Mixed-Integer Linear Programming (MILP) model. The notations used in the mathematical formulation are summarized in Table 2. The optimization constraints are defined as follows:

1. **Operation Assignment Constraint:** Each operation must be assigned to exactly one compatible machine.

$$\sum_{k \in \mathcal{M}} x_{i,j,k} = 1, \quad \forall i, j \quad (1)$$

2. **AGV Assignment Constraint:** Each transportation task between consecutive operations must be assigned to exactly one AGV.

$$\sum_{v \in \mathcal{V}} y_{i,j,v} = 1, \quad \forall i, j < n_i \quad (2)$$

3. **Timing and Coordination Constraints:** The start time of operation $O_{i,j+1}$ depends on the completion of $O_{i,j}$ and the transportation process. To explicitly model the synchronization between job availability and AGV arrival (including potential waiting times), the constraint is formulated as:

$$S_{i,j+1} \geq \max(C_{i,j}, A_v^{\text{arrival}}) + t_{\text{loaded}} \quad (3)$$

In the MILP formulation, this is linearized as:

$$S_{i,j+1} \geq C_{i,j} + \sum_{v \in \mathcal{V}} y_{i,j,v} \cdot (t_{\text{empty}} + t_{\text{loaded}}) \quad (4)$$

Note that t_{empty} accounts for the travel time from the AGV's previous location to the current pickup machine.

4. **Machine Disjunctive Constraints:** To ensure no two operations are processed on the same machine simultaneously, the standard Big-M formulation is applied. If operations $O_{i,j}$ and $O_{p,q}$ are assigned to machine k :

$$S_{p,q} \geq C_{i,j} - L \cdot (3 - x_{i,j,k} - x_{p,q,k} - z_{i,j,p,q}^{\mathcal{M}}) \quad (5)$$

$$S_{i,j} \geq C_{p,q} - L \cdot (2 + z_{i,j,p,q}^{\mathcal{M}} - x_{i,j,k} - x_{p,q,k}) \quad (6)$$

where $z_{i,j,p,q}^{\mathcal{M}}$ is a binary variable defining the processing sequence on the machine.

5. **AGV Disjunctive Constraints (Conflict-Free):** Similarly, an AGV cannot perform two transportation tasks simultaneously. If tasks for $O_{i,j}$ and $O_{p,q}$ are assigned to the same AGV v , their time windows must not overlap:

$$Start_{p,q}^{\text{trans}} \geq End_{i,j}^{\text{trans}} - L \cdot (3 - y_{i,j,v} - y_{p,q,v} - z_{i,j,p,q}^{\mathcal{V}}) \quad (7)$$

$$Start_{i,j}^{\text{trans}} \geq End_{p,q}^{\text{trans}} - L \cdot (2 + z_{i,j,p,q}^{\mathcal{V}} - y_{i,j,v} - y_{p,q,v}) \quad (8)$$

where $z_{i,j,p,q}^{\mathcal{V}}$ defines the transportation sequence on the AGV.

6. **Time Continuity Constraint:** This constraint enforces the temporal consistency of operations, strictly defining the completion time as the sum of the start time and the processing duration on the selected machine.

$$C_{i,j} = S_{i,j} + \sum_{k \in \mathcal{M}} x_{i,j,k} \cdot p_{i,j,k}, \quad \forall i, j \quad (9)$$

7. **AGV Workload Calculation:** To support the secondary objective of load balancing, the total workload of each AGV is defined as the total number of transportation tasks assigned to it.

$$W_v = \sum_{i \in \mathcal{J}} \sum_{j=1}^{n_i-1} y_{i,j,v} \cdot T_{i,j,v}^{\text{trans}}, \quad \forall v \in \mathcal{V} \quad (10)$$

Objective Function: This study considers a multi-objective optimization problem. The primary objective is to minimize the makespan (f_1), and the secondary objective is to minimize the workload variance among AGVs (f_2) to achieve load balancing:

$$\text{Minimize } F = \{f_1, f_2\} \quad (11)$$

where:

$$f_1 = C_{\max} = \max_{i \in \mathcal{J}} \{C_{i,n_i}\} \quad (12)$$

$$f_2 = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} (W_v - \bar{W})^2, \quad \text{with } \bar{W} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} W_v \quad (13)$$

Finally, the global objective function is formulated as a weighted sum of the efficiency and stability metrics:

$$\text{Minimize } Z = \omega_1 \cdot f_1 + \omega_2 \cdot f_2 \quad (14)$$

where ω_1 and ω_2 are weight coefficients representing the trade-off between production efficiency and workload balance.

The formulated model incorporates integer decision variables and non-linear objective components, such as load variance, rendering the problem strongly NP-hard. Due to this computational intractability, traditional exact solvers like CPLEX and Gurobi often fail to converge within a reasonable timeframe for large-scale instances. To address this scalability challenge, we adopt the hierarchical deep reinforcement learning approach detailed in Section 4.3 to efficiently learn near-optimal scheduling policies.

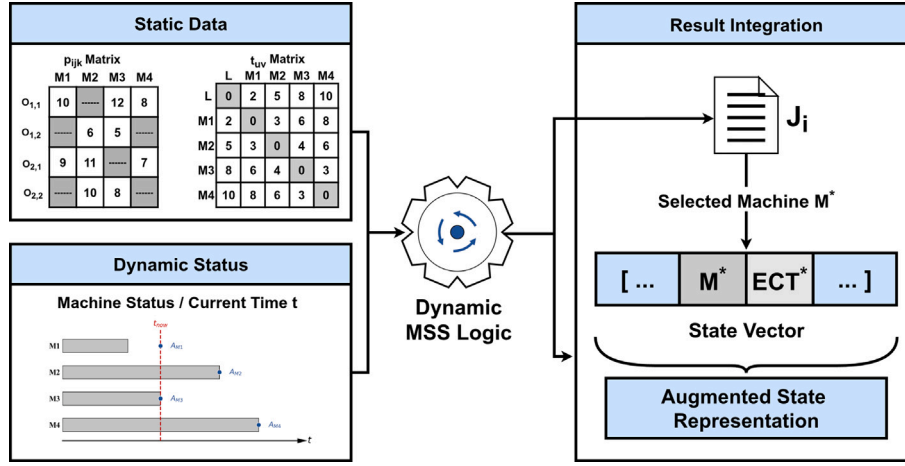


Fig. 3. Schematic framework of the Dynamic Heuristic-Based Machine Selection Strategy (MSS).

4.3. Heuristic-based machine selection

As formulated in the MILP model presented in Section 4.2, the simultaneous optimization of machine assignment, AGV dispatching, and operation sequencing results in an exponentially growing action space. To mitigate this complexity while retaining responsiveness to system dynamics, this study employs a **Dynamic Heuristic-Based Machine Selection Strategy (MSS)**.

As illustrated in Fig. 3, the MSS module functions as an embedded logic unit within the state generation process. The figure explicitly depicts the information flow: static parameters, specifically the processing and transportation matrices, combined with real-time machine availability ($A_k(t)$) serve as inputs to the dynamic logic, which generates the optimal machine index M^* for direct integration into the agent's state vector. Unlike static assignment rules that fix resources at the initial time, our strategy performs real-time machine assignment at each decision step based on the current system status. This explicitly decouples the machine assignment sub-problem from the RL agent's action space.

Specifically, for each operation $O_{i,j}$, the strategy evaluates all candidate machines to identify the one that offers the *Earliest Completion Time (ECT)* under the current workload distribution. Leveraging the processing time $p_{i,j,k}$ and the dynamic availability of resources, the estimated ECT on a candidate machine k is calculated as:

$$ECT_{i,j,k}(t) = \max(C_{i,j-1} + t_{loc(i),k}, A_k(t)) + p_{i,j,k} \quad (15)$$

In this equation, $C_{i,j-1}$ denotes the ready time of the job; $t_{loc(i),k}$ estimates the transportation time from the job's current location $loc(i)$ to machine k ; and $A_k(t)$ signifies the real-time available instant of machine k at decision step t . Consequently, the machine that minimizes this metric is dynamically selected:

$$M^*(t) = \arg \min_{k \in \mathcal{M}_{i,j}} \{ECT_{i,j,k}(t)\} \quad (16)$$

Crucially, regarding the integration with the state representation, this selection strategy is executed dynamically during the **State Tokenization** as detailed in Section 4.4. The attributes of the dynamically selected machine $M^*(t)$ – including its ID, estimated processing finish time, and the implied transportation cost – are encapsulated into the feature vector of the job. This allows the RL agent to perceive the best immediate machine option as a feature, thereby focusing its learning capacity on the higher-level logic of Job Sequencing and AGV Dispatching. The detailed procedural logic is outlined in Algorithm 1.

Algorithm 1 Dynamic Heuristic Machine Selection Strategy.

Require: Current Job State J_i , Current Time t , Machine Availability Set $\{A_k(t)\}$
Require: Static Parameters: Processing matrix p , Transport matrix $t_{u,v}$
Ensure: Selected machine M^* , Estimated completion time T^*

- 1: Initialize $T^* \leftarrow +\infty$
- 2: Get job ready time $T_{ready}^{job} \leftarrow$ Completion time of previous op $O_{i,j-1}$
- 3: **for** each candidate machine $k \in \mathcal{M}$ **do**
- 4: **if** $p_{i,j,k} < \infty$ **then**
- 5: $T_{travel} \leftarrow t_{loc(i),k}$
- 6: $T_{arrival} \leftarrow T_{ready}^{job} + T_{travel}$
- 7: $T_{ready}^{mach} \leftarrow A_k(t)$
- 8: $T_{ok} \leftarrow \max(T_{arrival}, T_{ready}^{mach}) + p_{i,j,k}$
- 9: **if** $T_{ok} < T^*$ **then**
- 10: $T^* \leftarrow T_{ok}$
- 11: $M^* \leftarrow k$
- 12: **end if**
- 13: **end if**
- 14: **end for**
- 15: **return** M^*, T^*

4.4. Hierarchical state encoding with unified tokens

In flexible manufacturing systems, the scale of the scheduling problem-specifically the number of active jobs and available AGVs-varies dynamically throughout the production lifecycle. Conventional reinforcement learning approaches constrained by fixed-length vector inputs, such as standard Multi-Layer Perceptrons (MLPs), struggle to accommodate this scalability. To address this limitation, this study adopts a Token-Based State Representation, treating discrete workshop entities as tokens in a sequence. This formulation leverages the Transformer architecture's inherent capability to process variable-length sets efficiently.

A critical challenge in constructing a unified state space is the heterogeneity of workshop entities: Jobs possess complex attribute sets, such as operation progress, urgency, and heuristic preferences, whereas AGVs are characterized by simpler logistics metrics. To map these distinct entities into a shared vector space compatible with the Transformer's input requirements, we establish a unified feature dimension D_{feat} (set to 24 in this study). As visually detailed in Fig. 4, which depicts the transformation from raw inputs to the unified state matrix, we employ a zero-padding alignment mechanism where Job tokens (v_j) containing 20 dimensions of attributes and AGV tokens (v_A) containing 7 dimensions are both padded with zeros to align with D_{feat} . This strategy ensures that heterogeneous entities can be stacked into a homogeneous mathematical structure without any information loss.

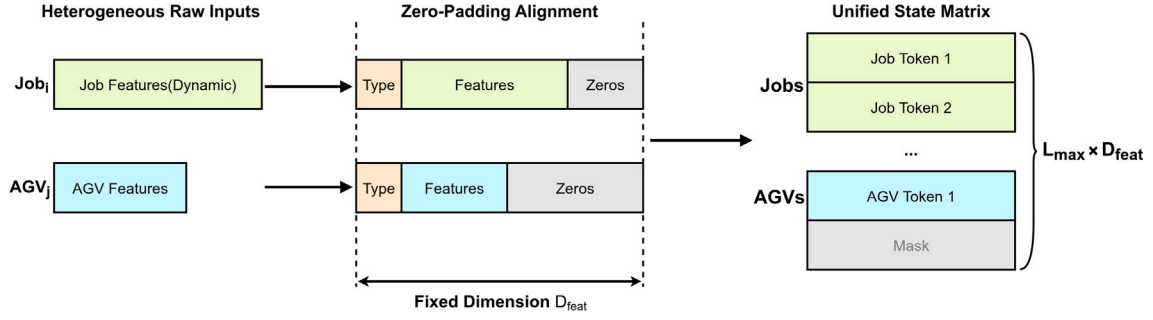


Fig. 4. Schematic of the unified state sequence construction.

Building upon this unified feature representation, the system must further address the dynamic variation in the number of entities to facilitate batch processing in deep neural networks. Although the token count fluctuates during production, the state input requires a static tensor shape. To accommodate this, we introduce a maximum sequence length, L_{max} , serving as an upper bound capacity. Consequently, the global state at time step t is formulated as a composite sequence $S_t \in \mathbb{R}^{L_{max} \times D_{feat}}$:

$$S_t = \text{Concat}(\underbrace{v_{J_1}, \dots, v_{J_n}}_{\text{Active Jobs}}, \underbrace{v_{A_1}, \dots, v_{A_m}}_{\text{AGVs}}, \underbrace{v_{pad}, \dots}_{\text{Buffer Padding}}) \quad (17)$$

where v_{pad} denotes entirely masked tokens. These validity masks are passed to the Attention mechanism to ensure the agent ignores empty buffer slots during computation.

Unlike natural language processing where the sequential order of words determines semantic meaning, the set of jobs in a workshop exhibits permutation invariance. This characteristic implies that the scheduling policy must depend on the intrinsic attributes of a job rather than its index in the input list. Consequently, we explicitly discard standard sinusoidal positional encoding to prevent the model from overfitting to artificial sequence orders. To compensate for the absence of sequential context, we introduce distinct auxiliary identifications. Specifically, a one-hot entity type header is prepended to each token to strictly differentiate between processing tasks and logistics resources. Furthermore, dynamic interaction flags including the distance to the target job and estimated arrival time are embedded directly into the AGV tokens during the decision phase to guide the agent's focus. The comprehensive composition of these aligned feature vectors is detailed in Table 3.

4.5. Dual-agent policy and decision mechanism

To efficiently model the complex coupling between operation sequencing and resource allocation, this study proposes a hierarchical collaborative decision-making mechanism. As illustrated in the overall framework, the scheduling problem is decomposed into two sequential sub-tasks handled by independent agents: a Job Agent responsible for operation selection and an AGV Agent dedicated to vehicle dispatching. This decomposition reduces the combinatorial complexity of the joint action space while maintaining the logical dependency between task scheduling and logistics.

Central to this framework is the shared neural network architecture depicted in Fig. 5, tailored for the token-based state space $S_t \in \mathbb{R}^{L_{max} \times D_{feat}}$. The architecture integrates two key components optimized for set-based processing to facilitate end-to-end training via the Proximal Policy Optimization (PPO) algorithm. On the policy side, the Actor network (left in Fig. 5) employs a multi-layer Transformer Encoder to extract contextual relationships between heterogeneous tokens. The resulting embeddings are processed through a specific masking layer—which filters out invalid entities such as completed jobs or padding—before entering the Softmax layer to generate a valid probability distribution.

Table 3

Detailed definition of unified state features (dimension $D_{feat} = 24$).

Token type	Idx	Feature notation	Mathematical definition / description
Job Token (v_J)	1–2	Entity Type	$e_{type} = [1, 0]$ (One-hot indicator)
	3	Selection Flag	$\mathbb{I}(J_i \text{ is selected})$ (Indicator function: 1 if selected, 0 otherwise)
	4	Progress Ratio	j/n_i (Current operation index / Total operations)
	5	Remain Ops	$(n_i - j)/N_{cap}$ (Normalized remaining operations)
	6	MSS Choice	M^* (Optimal machine derived from Eq. (16))
	7	Proc Time	$p_{i,j,M^*}/T_{norm}$ (Norm. processing time on M^*)
	8	Trans Time	$t_{loc(i),M^*}/T_{norm}$ (Norm. travel time from current loc to M^*)
	9	ECT	$ECT_{i,j,M^*}/T_{norm}$ (From Eq. (15))
	10	Global Remain	$T_{remain}^{est}/T_{norm}$ (Estimated remaining time for whole job)
	11	End Time	$C_{i,j-1}/T_{norm}$ (Job ready time / Previous op finish time)
	12	Last Time	$C_{i,j-1}/T_{norm}$ (Redundant with above, strictly means $C_{i,j-1}$)
	13	Continuity	$\mathbb{I}(Loc(J_i) == Loc(M^*))$
	14	Machine Wait	$\max(0, A_{M^*}(t) - C_{i,j-1})/T_{norm}$
	15	Flexibility	$ M_{i,j} /m$ (Available machines / Total machines)
	16	Advantage	Heuristic score relative to average processing time
	17	Min AGV Cost	$\min_{v \in V} (\text{dist}(v, J_i))/D_{norm}$
	18	Urgency	$(t + T_{remain}^{est})/C_{max}^{est}$ (Ratio to estimated Makespan)
	19–24	Padding	$\mathbf{0}$ vector to align dimension to D_{feat}
AGV Token (v_A)	1–2	Entity Type	$e_{type} = [0, 1]$ (One-hot indicator)
	3	Position	$Loc(v)/m$ (Norm. current location node index)
	4	Avail Time	t_{free}^v/T_{norm} (Time when AGV becomes idle)
	5	Dist to Target	$\text{dist}(Loc(v), Loc(J_{target}))/D_{norm}$
	6	Arrival Time	$(t + t_{travel})/T_{norm}$
	7	Same Loc	$\mathbb{I}(Loc(v) == Loc(J_{target}))$
	8–24	Padding	$\mathbf{0}$ vector to align dimension to D_{feat}

Note: $\mathbb{I}(\cdot)$ denotes the indicator function. n_i follows the definition in Table 2. $Loc(\cdot)$ denotes the current node index. N_{cap} , T_{norm} , D_{norm} are normalization constants for maximum scale, time, and distance, respectively.

Complementing the policy generation, the Critic network (right in Fig. 5) estimates the state value $V(S_t)$ by aggregating encoded token features into a global representation. This is achieved using a row-sum pooling operation followed by a Multi-Layer Perceptron (MLP), ensuring that the value baseline effectively captures the global workload status while remaining invariant to the permutation of job tokens.

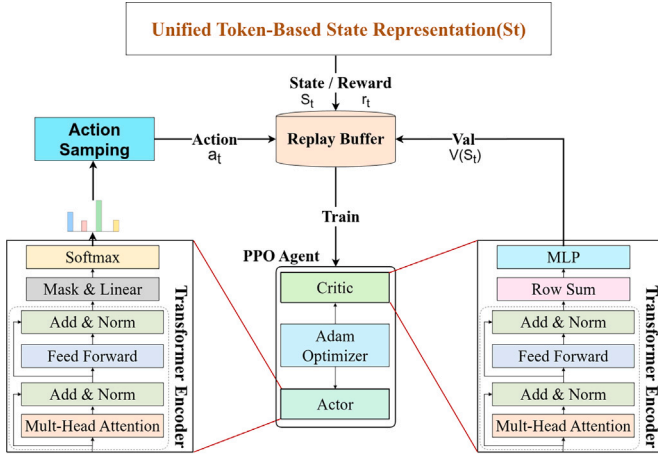


Fig. 5. Network architecture of the Transformer-based PPO agent.

4.5.1. Hierarchical action space and masking

Unlike simultaneous action generation, our approach adopts a conditional sequential execution flow where the joint action $a_t = (a_t^{job}, a_t^{agv})$ is derived in stages. The process commences with the high-level Job Agent observing the global state S_t to select the index of the next job. Since the policy network outputs a distribution over the entire input sequence, the raw action space corresponds to the token indices:

$$\mathcal{A}_{job} = \{1, 2, \dots, L_{max}\} \quad (18)$$

To strictly confine the policy exploration within the feasible domain, a validity mask M_t^{job} is applied to the output logits. This mask forces the probabilities of invalid actions – specifically, completed jobs and zero-padding tokens:

$$P(a_t^{job} = i | S_t) = \text{Softmax}(\text{Logits}_i + M_t^{job}[i]) \quad (19)$$

where $M_t^{job}[i]$ is set to 0 if job J_i is unfinished and schedulable, and $-\infty$ otherwise.

Upon the selection of a specific job J_i , the environment performs a virtual state transition via the heuristic MSS logic to determine the target machine M^* . Critically, the state is updated to an intermediate context S'_t , where the dynamic interaction features of all AGV tokens (e.g., distance to J_i , estimated arrival time) are re-calculated relative to the selected task. Conditioned on this updated context, the low-level AGV Agent selects a vehicle token from the sequence:

$$\mathcal{A}_{agv} \subseteq \{1, 2, \dots, L_{max}\} \quad (20)$$

Analogous to the job selection phase, an AGV mask M_t^{agv} is employed to strictly restrict the probability distribution to valid AGV tokens, effectively masking out job and padding entries. Through this hierarchical decomposition, the action space complexity is effectively reduced from combinatorial $\mathcal{O}(|J| \times |\mathcal{V}|)$ to linear $\mathcal{O}(|J| + |\mathcal{V}|)$, significantly accelerating the convergence of the learning process.

4.5.2. Hybrid reward function

To facilitate the learning of efficient joint scheduling strategies, this study designs a hierarchical reward structure aligned with the two-level decision framework. The primary objective is to minimize the global makespan while optimizing logistics efficiency. However, relying solely on the final makespan as a sparse reward signal often leads to convergence difficulties in long-horizon tasks. To address this, we propose a hybrid reward mechanism that integrates dense step-wise penalties for immediate operational feedback and sparse terminal incentives for global quality evaluation. This structure is further explicitly decomposed

to reinforce the distinct responsibilities of the Job Agent and the AGV Agent.

During the decision-making process, the system calculates a dense immediate reward r_t at each time step t to guide agents toward local efficiency. This dense signal is constructed as a weighted sum of three penalty components: transportation cost, waiting cost, and makespan increment. Specifically, the transportation cost r_{trans} is defined to penalize long-distance dispatching, formulated as:

$$r_{trans} = -\lambda_1 \cdot T_{trans} \quad (21)$$

where T_{trans} represents the actual travel time of the selected AGV and λ_1 is a scaling coefficient. By applying this negative feedback, the algorithm encourages the AGV agent to prioritize vehicles in close spatial proximity to the target job, thereby enhancing logistical responsiveness. Simultaneously, to improve the synchronization between processing and logistics, a waiting cost r_{wait} is introduced:

$$r_{wait} = -\lambda_2 \cdot T_{wait} \quad (22)$$

where T_{wait} quantifies the non-value-added time caused by scheduling misalignments, such as an AGV waiting for a machine to become available. Furthermore, to align local step-wise decisions with the global optimization objective, a makespan increment penalty is applied:

$$r_{\Delta C_{max}} = -\lambda_3 \cdot \max(0, C_{max}(t) - C_{max}(t-1)) \quad (23)$$

Here, $C_{max}(t)$ denotes the global makespan after step t . This term provides a continuous gradient signal that forces the agents to prioritize critical path operations, minimizing the extension of the total production cycle.

Upon the completion of all jobs, a sparse terminal reward R_{done} is granted to evaluate the final solution quality. This signal comprises a fixed completion bonus ξ_1 to ensure successful episode termination and a load balancing penalty to mitigate resource bottlenecks. The load balancing term is formulated based on the standard deviation of the workload distribution:

$$r_{bal} = -\xi_2 \cdot \sigma(\mathcal{W}_{agv}) \quad (24)$$

where $\mathcal{W}_{agv} = \{w_1, w_2, \dots, w_{|V|}\}$ represents the set of task counts assigned to each AGV. Minimizing the standard deviation $\sigma(\cdot)$ promotes a more equitable distribution of transport tasks across the fleet, preventing the overuse of specific vehicles and enhancing system robustness.

Finally, to address the credit assignment problem in multi-agent co-operation, the aggregated rewards are decomposed according to the specific roles of each agent. The Job Agent, primarily responsible for operation sequencing, receives a reward R_{Job} focusing on process synchronization and makespan reduction. In contrast, the AGV Agent, which manages physical logistics, receives a comprehensive reward R_{AGV} that includes transportation efficiency and load balancing. The agent-specific reward functions are formally defined as:

$$\begin{aligned} R_{Job} &= r_{wait} + r_{\Delta C_{max}} + \xi_1 \\ R_{AGV} &= r_{trans} + r_{wait} + r_{\Delta C_{max}} + r_{bal} + \xi_1 \end{aligned} \quad (25)$$

This decomposition ensures that the Job Agent focuses on optimizing the production rhythm, while the AGV Agent bears the additional responsibility for minimizing transport costs and maintaining resource fairness.

4.5.3. Policy optimization algorithm

Based on the hierarchical architecture and the hybrid reward mechanism defined above, the entire system is trained end-to-end using the Proximal Policy Optimization (PPO) algorithm. Since the Job Agent and AGV Agent operate with distinct action spaces and receive differentiated

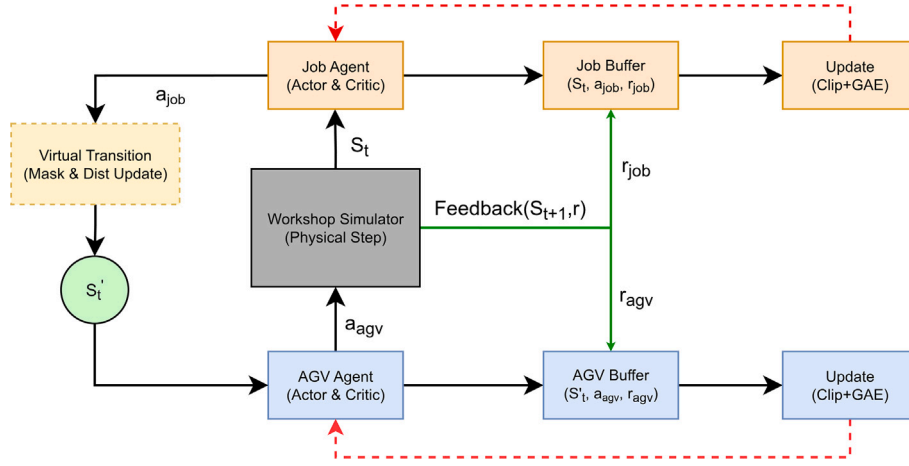


Fig. 6. Dual-Agent Cooperative Training Framework.

reward signals, we maintain two independent sets of network parameters, denoted as θ_{job} and θ_{agv} , along with their respective Critics ϕ_{job} and ϕ_{agv} . These networks are updated simultaneously but independently, allowing each agent to specialize in its specific decision domain while coordinating through the shared environment dynamics.

The training process follows a conditional sequential interaction protocol, as visually outlined in Fig. 6. At the beginning of each decision step t , the Job Agent evaluates the global state S_t to determine the initial job selection action a_t^{job} . Upon this selection, the environment initiates a virtual transition to an intermediate state S'_t , efficiently incorporating target machine assignments and updated AGV logistics features. Conditioned on this intermediate context, the AGV Agent subsequently generates the dispatching action a_t^{agv} . Once the joint action is fully formed, the environment executes the physical step, yielding the next global state S_{t+1} alongside agent-specific rewards r_t^{job} and r_t^{agv} . To support decoupled credit assignment, these transition tuples are organized into separate experience buffers, corresponding to the distinct data flows illustrated in the right panel of Fig. 6.

To reduce the variance of gradient estimation during updates, we employ the Generalized Advantage Estimation (GAE) method. For each agent, the advantage $\hat{A}_t$ is computed using the temporal difference error δ_t :

$$\delta_t = r_t + \gamma V_\phi(S_{t+1}) - V_\phi(S_t) \quad (26)$$

$$\hat{A}_t = \sum_{k=0}^{T-t-1} (\gamma \lambda)^k \delta_{t+k} \quad (27)$$

where γ is the discount factor and λ is the GAE smoothing parameter. The policy parameters are updated by maximizing the clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (28)$$

where $\rho_t(\theta)$ is the probability ratio between the new and old policies, and ϵ limits the update magnitude. The total loss function includes a value function loss L^{VF} and an entropy bonus H to encourage exploration, formulated as $L_{total} = -L^{CLIP} + c_1 L^{VF} - c_2 H$. The detailed training procedure is summarized in Algorithm 2.

5. Experiments and results

To systematically evaluate the effectiveness and robustness of the proposed MA-Trans framework in flexible job shop environments with AGV constraints, we conducted a comprehensive experimental study. The evaluation begins with a comparative performance analysis against state-of-the-art graph-based and recurrent deep reinforcement learning

Algorithm 2 Dual-Agent Cooperative PPO Training Algorithm.

Require: Environment $\mathcal{E}$, Job Agent parameters θ_{job} , AGV Agent parameters θ_{agv}

Require: Hyperparameters: Learning rate α , Clip ϵ , GAE λ , Horizon T

```

1: for episode = 1 to  $N_{episodes}$  do
2:   Reset environment to get initial state  $S_t$ 
3:   while not Done do
4:     Generate job action  $a_t^{job}$  from  $\pi_{\theta_{job}}(S_t)$  with validity mask
5:     Execute virtual transition:  $S'_t, \text{Mask}^{agv} \leftarrow \mathcal{E}.\text{job\_step}(a_t^{job})$ 
6:     Generate AGV action  $a_t^{agv}$  from  $\pi_{\theta_{agv}}(S'_t)$  with mask
7:     Execute physical transition:  $S_{t+1}, r_t^{job}, r_t^{agv}, \text{Done} \leftarrow \mathcal{E}.\text{step}(a_t^{job}, a_t^{agv})$ 
8:     Store  $(S_t, a_t^{job}, r_t^{job}, \text{Done})$  in Job Buffer  $B_{job}$ 
9:     Store  $(S'_t, a_t^{agv}, r_t^{agv}, \text{Done})$  in AGV Buffer  $B_{agv}$ 
10:    Update state  $S_t \leftarrow S_{t+1}$ 
11:  end while
12:  if update condition met then
13:    for agent  $k \in \{job, agv\}$  do
14:      Compute Advantages  $\hat{A}$  using GAE on Buffer  $B_k$ 
15:      Update  $\theta_k$  by maximizing  $L^{CLIP}$  via Adam optimizer
16:      Clear Buffer  $B_k$ 
17:    end for
18:  end if
19: end for

```

baselines, specifically ResGAT and LSTM-Ptr, within a standardized simulation setup. Following this, we assess the zero-shot generalization capability of the model by applying trained policies to unseen problem scales without further fine-tuning. To provide a holistic view of practical applicability, we then analyze computational efficiency, focusing on the trade-off between model complexity and inference latency. Finally, the chapter concludes with a visualization case study using Gantt charts to offer intuitive insights into the interpretable coordination mechanisms between machines and AGVs.

5.1. Experimental setup

5.1.1. Dataset generation

To evaluate the performance of the proposed method, we constructed a benchmark dataset based on the widely used Flexible Job Shop Scheduling Problem instances open-sourced by Song et al [25]. Since the original dataset focuses solely on machine processing constraints without considering logistic factors, it lacks the necessary configurations for AGV transportation. To bridge this gap and accurately simulate the dual-resource constraints characteristic of FJSP-AGV systems, we extended the original instances by explicitly configuring the AGV fleet size and generating corresponding transportation time matrices.

The extended dataset encompasses four representative problem scales, denoted as $n \times m \times v$ (jobs $\times$ machines $\times$ AGVs). A critical feature of these datasets is the Logistics-Intensive setting, where the average

transportation time is comparable to the processing time, approximating a 1:1 ratio. This setup introduces strong spatio-temporal coupling, requiring the scheduling algorithm to simultaneously optimize machine assignment and AGV routing. The scales are categorized to examine the algorithm under varying degrees of complexity and resource constraints:

- *Small Scale / Validation* ($10 \times 5 \times 2$): A baseline scenario with 10 jobs, 5 machines, and 2 AGVs. This setting serves as the primary environment to validate the fundamental logic learning capabilities of the agents without severe resource contention.
- *Medium Scale / Balanced* ($15 \times 10 \times 4$): A standard scenario representing a balanced workload. With a moderate job-to-machine ratio and sufficient AGV configuration, this scale is used to evaluate the algorithm's stability and efficiency in handling routine production tasks.
- *Medium Scale / Resource-Constrained* ($20 \times 5 \times 3$): A highly constrained scenario characterized by a high job-to-machine ratio (4:1) and a scarce AGV fleet. This setting simulates a bottleneck environment where the system degrades into a complex queuing problem, testing the algorithm's ability to maximize resource utilization under extreme pressure.
- *Large Scale / High-Density* ($20 \times 10 \times 5$): A complex scenario comprising 20 jobs, 10 machines, and 5 AGVs. This setting represents a manufacturing environment with high logistic density and long-horizon dependencies, requiring precise coordination between the AGV fleet and the machining system to minimize delays caused by traffic congestion and task coupling.

To ensure a rigorous evaluation, for each of the defined scales, we generated a fixed validation set of 100 instances for model checkpoint selection and a separate test set of 100 instances for final performance assessment. Regarding the transportation configuration, we generated a symmetric transportation time matrix for each workshop scale. The transfer times between different machines were randomly sampled from a uniform distribution with an upper bound that scales with the workshop size. This design ensures that logistic times remain comparable to processing times, preventing one factor from dominating the makespan. It is important to note that both the validation and test sets are strictly isolated from the training process to accurately evaluate the zero-shot generalization capability of the proposed model.

5.1.2. Baseline algorithms

To comprehensively evaluate the performance of the proposed MA-Trans framework, we compare it against six baseline algorithms selected to benchmark solution quality, computational efficiency, and architectural superiority. To benchmark against traditional constructive heuristics, we employ two widely recognized dispatching rules: MWKR-EAT and G-EST [34]. Unlike simple stochastic baselines, these deterministic rules represent competitive strategies for global workload balancing and local greedy optimization, respectively. Including them allows us to assess whether the reinforcement learning agent has surpassed the capabilities of established industrial-standard rules.

In contrast to these simple heuristics, we introduce the *Memetic Algorithm (MA)* to serve as a rigorous benchmark for solution quality. By integrating a hill-climbing local search mechanism with global evolutionary operators, MA is capable of deeply exploiting the solution space to effectively escape local optima [29]. Although computationally intensive, this hybrid characteristic allows it to provide a robust upper bound for scheduling performance, serving as a formidable competitor for evaluating the optimality gap.

Furthermore, to distinguish the specific advantages of the proposed Transformer architecture and the hierarchical mechanism, we conduct a comparative analysis against three state-of-the-art Deep Reinforcement Learning (DRL) baselines. These include a graph-based model (*ResGAT*), a recurrent model (*LSTM-Ptr*), and an ablation variant (*SA-PPO*). To ensure a strictly fair comparison regarding representational capacity,

all DRL baselines are implemented with the same embedding dimension ($d_{model} = 128$) and trained using the PPO algorithm under identical hyperparameters.

The specific details of the six baseline algorithms are as follows:

- *MWKR-EAT*: A representative rule for global workload balancing that prioritizes the job with the most work remaining. By processing tasks with the longest total remaining processing time first, this rule effectively mitigates the long tail effect in the final schedule and balances machine utilization. Once a job is selected, it is assigned to the machine that offers the earliest completion time, and the AGV is dispatched based on the earliest arrival time strategy [34].
- *G-EST*: A composite greedy rule that simultaneously evaluates all candidate job-machine pairs to find the one with the global earliest start time. Unlike sequential rules that select a job first and then a machine, G-EST considers the coupling of AGV transport and machine processing to minimize the immediate completion time of the next operation, serving as a strong baseline for local optimization performance.
- *MA*: A high-performance Memetic Algorithm (MA) is adopted as a strong baseline, integrating the global search capability of evolutionary computation with a fine-grained local search mechanism. Specifically, the algorithm utilizes Precedence Operation Crossover (POX) for operation sequencing and Uniform Crossover for machine assignment to maintain population diversity. Crucially, to overcome the premature convergence often observed in traditional evolutionary methods, a Variable Neighborhood Search (VNS) operator is selectively applied to the top-tier elite individuals in each generation. This hybrid intensification strategy significantly refines the solution quality by exploring the neighborhood of high-quality candidates. The implementation follows the robust architecture described in [29], tailored for the complex multi-shop scheduling problem with AGV constraints.
- *ResGAT*: A state-of-the-art graph-based baseline that utilizes Graph Attention Networks (GAT) [25] with residual connections to capture the topological structure of the scheduling graph. To ensure a fair comparison, we integrated the ResGAT encoder into the same PPO-based Actor-Critic framework as our method, replacing the Transformer module. This baseline aggregates information from local neighbors to model operation precedence, allowing us to explicitly evaluate the advantage of the Transformer's global attention mechanism over graph-based message passing in handling long-range dependencies and large-scale variability.
- *LSTM-Ptr*: A robust sequence-to-sequence baseline that employs a Bidirectional LSTM encoder coupled with a Pointer Network decoder. This architecture is widely regarded as a standard benchmark in neural combinatorial optimization for capturing sequential dependencies. To ensure a rigorous comparison, we aligned the model capacity of LSTM-Ptr with our proposed method by configuring it with the same number of layers (3 stacked layers) and hidden embedding dimensions (128) as our Transformer model. This configuration allows us to explicitly evaluate the architectural advantage of the self-attention mechanism over recurrent neural networks under comparable parameter complexity [15].
- *SA-PPO*: An ablation variant of our proposed method. It employs the identical Transformer-based policy for the Job Agent but replaces the learned AGV Agent with a greedy heuristic based on the Nearest Vehicle Rule. This baseline is critical to verify the necessity of the hierarchical structure and to demonstrate the contribution of the collaborative AGV policy.

5.1.3. Hyperparameter settings and experimental environment

The scheduling policy in this study is trained using the Proximal Policy Optimization (PPO) algorithm, implemented with the PyTorch

Table 4
Hyperparameter settings for MA-trans.

Category	Parameter	Value
<i>Training Configuration</i>	Total Episodes	50,000 – 100,000
	Batch Size	128
	Optimizer	Adam
	Learning Rate (LR)	2×10^{-4}
	LR Scheduler	Linear Decay (1.0 $\rightarrow$ 0.1)
<i>PPO Hyperparameters</i>	Discount Factor (γ)	0.99
	GAE Parameter (λ)	0.95
	Clip Range (ϵ)	0.2
	Update Epochs (K)	5
	Entropy Coeff. (β)	Linear Decay (0.05 $\rightarrow$ 0.01)
<i>Network Architecture</i>	Input Dimension	24
	Embedding Size (d_{model})	128
	Transformer Layers	3
	Attention Heads	4
	Dropout Rate	0.1

framework. The detailed hyperparameter configurations are derived from extensive preliminary tuning and are summarized in Table 4.

During the training phase, we employ the Adam optimizer for both the Actor and Critic networks with a unified initial learning rate of 2×10^{-4} . To stabilize the convergence process, a Linear Learning Rate Scheduler is applied, which linearly decays the learning rate from its initial value to 10% over the course of total training updates. The discount factor γ is set to 0.99 to encourage long-term planning, while the Generalized Advantage Estimation (GAE) parameter λ is set to 0.95 to balance bias and variance. The policy updates utilize a clipped surrogate objective with a clipping threshold $\epsilon = 0.2$. To prevent premature convergence, we incorporate an entropy regularization term with a coefficient that linearly decays from 0.05 to 0.01. The training process spans from 50,000 to 100,000 episodes depending on the problem scale, with a batch size of 128. For each update iteration, the networks are optimized over 5 epochs.

Regarding the network architecture, the raw state input dimension is 24, representing the 24 distinct features of jobs and AGVs defined in the environment. The Transformer encoder consists of 3 stacked layers with 4 attention heads and a hidden embedding dimension of 128 ($d_{model} = 128$). A dropout rate of 0.1 is applied to prevent overfitting.

The experiments are conducted on a workstation running Ubuntu 24.04.3 LTS, equipped with an Intel Core i7-14700 processor, 32 GB of RAM, and an NVIDIA GeForce RTX 4060 GPU (8 GB). This hardware configuration provides sufficient computational power to support the parallel sampling and gradient updates required for large-scale scheduling training.

For the comparative baselines, to ensure a fair and rigorous comparison, the hyperparameters of the MA are adaptively scaled according to the problem complexity. The population size is configured between 100 and 200, while the number of generations ranges from 100 to 300 to guarantee sufficient convergence on large-scale instances [3]. Furthermore, considering the stochastic variability inherent in evolutionary algorithms, each test instance is evaluated over 5 independent runs, and the mean makespan is reported as the final performance metric to eliminate random bias.

5.2. Comparative performance analysis

Table 5 presents the quantitative results of the proposed MA-Trans framework and six baseline algorithms across four validation scenarios ranging from small-scale to large-scale instances. The evaluation metrics include the mean and standard deviation of the makespan derived from 100 independent test instances. To explicitly measure the performance advantage, the relative optimality gap is reported with respect to our method. Furthermore, a paired t -test is conducted to verify statistical significance, with p -values reported for each pairwise comparison to ensure the reliability of the improvements.

Table 5
Performance comparison and statistical significance analysis on four test scenarios ($N = 100$).

Scale	Method	Mean $\pm$ Std	Gap (%)	p -value
$10 \times 5 \times 2$	G-EST	272.8 $\pm$ 31.9	+27.4	< 0.001
	MWKR-EAT	253.6 $\pm$ 25.3	+18.5	< 0.001
	MA	244.4 $\pm$ 18.8	+14.2	< 0.001
	SA-PPO	248.3 $\pm$ 30.4	+16.0	< 0.001
	ResGAT	221.2 $\pm$ 22.7	+3.3	< 0.001
	LSTM-Ptr	225.7 $\pm$ 22.0	+5.5	< 0.001
	MA-Trans (Ours)	214.1 $\pm$ 20.8	–	–
$15 \times 10 \times 4$	G-EST	377.7 $\pm$ 37.6	+27.5	< 0.001
	MWKR-EAT	334.4 $\pm$ 21.7	+12.9	< 0.001
	MA	459.7 $\pm$ 21.9	+55.2	< 0.001
	SA-PPO	365.9 $\pm$ 29.5	+23.6	< 0.001
	ResGAT	320.1 $\pm$ 22.4	+8.1	< 0.001
	LSTM-Ptr	305.3 $\pm$ 23.6	+3.1	< 0.001
	MA-Trans (Ours)	296.2 $\pm$ 18.8	–	–
$20 \times 5 \times 3$	G-EST	360.3 $\pm$ 27.6	+32.5	< 0.001
	MWKR-EAT	355.5 $\pm$ 22.0	+30.7	< 0.001
	MA	352.6 $\pm$ 18.1	+29.7	< 0.001
	SA-PPO	355.3 $\pm$ 40.9	+30.7	< 0.001
	ResGAT	277.7 $\pm$ 15.4	+2.1	< 0.001
	LSTM-Ptr	275.1 $\pm$ 15.5	+1.2	0.021
	MA-Trans (Ours)	271.9 $\pm$ 16.6	–	–
$20 \times 10 \times 5$	G-EST	421.1 $\pm$ 34.6	+29.9	< 0.001
	MWKR-EAT	371.5 $\pm$ 19.4	+14.6	< 0.001
	MA	524.4 $\pm$ 20.0	+61.7	< 0.001
	SA-PPO	419.0 $\pm$ 30.1	+29.2	< 0.001
	ResGAT	347.2 $\pm$ 21.1	+7.1	< 0.001
	LSTM-Ptr	335.3 $\pm$ 21.5	+3.4	< 0.001
	MA-Trans (Ours)	324.2 $\pm$ 18.3	–	–

Note: 1. **G-EST**: Global Earliest Start Time rule. 2. **MWKR-EAT**: Most Work Remaining job selection with Earliest Arrival Time dispatching. 3. **MA**: Memetic Algorithm. 4. **SA-PPO**: Single-Agent PPO. 5. **Gap**: Calculated as $(Mean_{Base} - Mean_{Ours}) / Mean_{Ours} \times 100\%$. 6. **p -value**: Calculated via paired t -test against MA-Trans. Values < 0.05 indicate statistical significance.

Traditional heuristics, namely G-EST and MWKR-EAT, exhibit substantial performance deficits, lagging behind MA-Trans by approximately 15% to 30%. This confirms that myopic dispatching rules fail to handle the complex coupling between AGV transport and machine processing effectively. Similarly, the reinforcement learning baseline SA-PPO performs comparably to these rule-based methods. This limited performance indicates that decomposing the problem into a single agent coupled with heuristic logic disrupts the joint optimization goal, leading to sub-optimal policies. Regarding the evolutionary baseline, while MA achieves competitive results on small scales, its performance deteriorates significantly on larger instances, reaching a gap of 61.7% on the $20 \times 10 \times 5$ dataset. This degradation is attributed to the exponential growth of the search space, where the iterative search mechanism of MA struggles to converge to global optima within a limited time budget compared to the direct inference capability of the learned policy.

Among the baselines, ResGAT and LSTM-Ptr demonstrate the strongest competitiveness, maintaining a relatively narrow margin within 8% of our method. However, MA-Trans consistently achieves the lowest makespan across all scenarios. A distinct phenomenon arises in the $20 \times 5 \times 3$ scenario, which represents a resource-constrained environment with few machines and AGVs relative to the number of jobs. In this specific context, the problem effectively degrades into a tight queuing system where the room for optimization is inherently limited. Consequently, the performance advantage narrows, with LSTM-Ptr achieving a gap of only 1.2%. The standard deviation of our method also becomes comparable to the baselines, reflecting the high variance inherent in such constrained conditions. Nevertheless, the paired t -test yields a p -value of 0.021, confirming that even in this highly restricted scenario, the advantage of MA-Trans remains statistically significant despite the reduced margin compared to resource-sufficient scenarios.

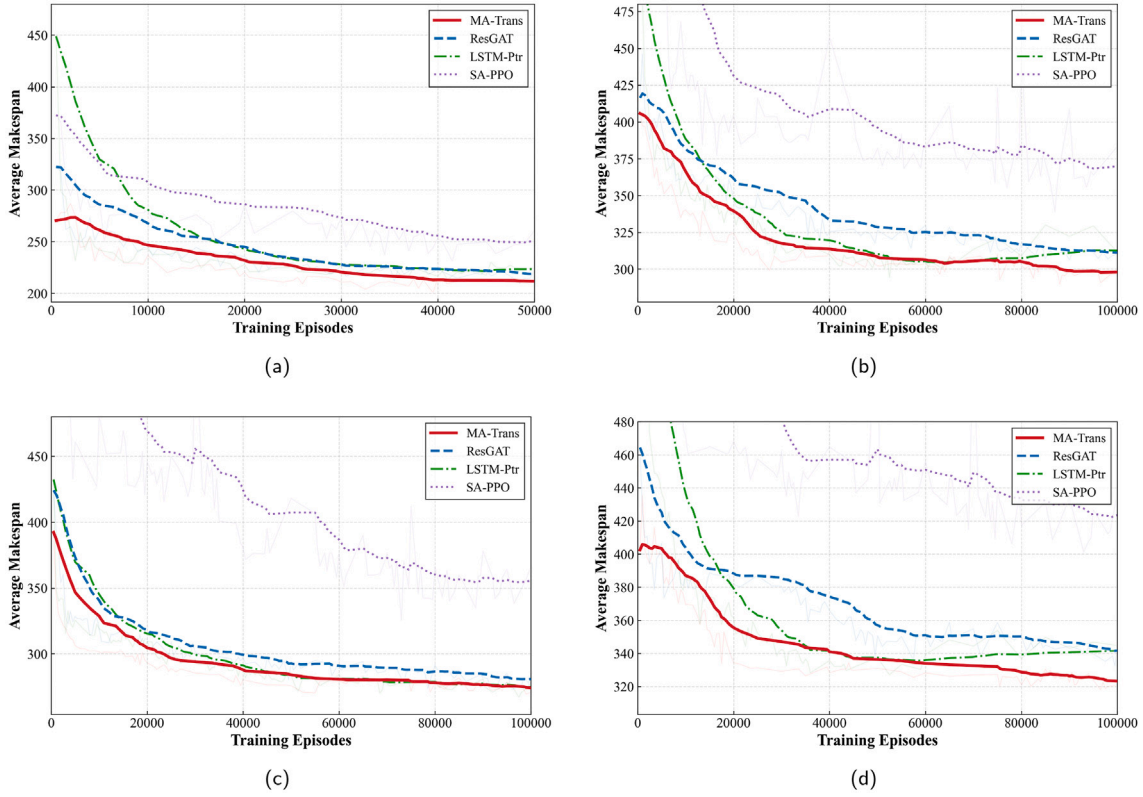


Fig. 7. Comparative analysis of average makespan convergence trends among DRL-based scheduling algorithms across four distinct scenarios – (a) Small-Scale Validation ($10 \times 5 \times 2$), (b) Medium-Scale Balanced ($15 \times 10 \times 4$), (c) Resource-Constrained Bottleneck ($20 \times 5 \times 3$), (d) Large-Scale High-Density ($20 \times 10 \times 5$).

To further investigate the learning efficiency and stability of the proposed framework, Fig. 7 illustrates the convergence curves of the average makespan throughout the training phase, where the duration is set to 50,000 episodes for the small-scale scenario and extended to 100,000 episodes for larger complex scenarios to ensure sufficient policy maturation.

Consistently exhibiting the lowest initial starting point and the fastest convergence rate across all scenarios, MA-Trans benefits from the powerful global feature extraction and self-attention mechanism of the Transformer. This allows the agent to effectively capture the complex coupling between jobs and AGVs from the early stages. For instance, in Fig. 7(a), the initial performance of MA-Trans is equivalent to the level achieved by other baselines after 10,000 to 20,000 training episodes, demonstrating a significant early advantage in policy optimization. Conversely, SA-PPO displays the poorest convergence. As the problem scale increases, the conflict between the learned job selection policy and the fixed AGV dispatching rule intensifies, leading to a progressively widening gap compared to the end-to-end approaches.

A critical phenomenon appears in the medium and large-scale scenarios shown in Fig. 7(b) and Fig. 7(d), where the convergence curve of LSTM-Ptr briefly overlaps with that of MA-Trans around 50,000 to 60,000 episodes. However, due to the inherent forgetting issue of recurrent networks and their short-sightedness in handling long sequences, LSTM-Ptr tends to get trapped in local optima and ceases to improve. In contrast, MA-Trans continues to explore and optimize, eventually breaking through the bottleneck to achieve a lower asymptotic makespan. It is also observed that ResGAT generally underperforms compared to LSTM-Ptr in these large-scale scenarios, suggesting that while GNNs capture topological dependencies, the sequential decision-making nature of scheduling is better modeled by sequence-based architectures like LSTM and Transformer, whereas ResGAT may suffer from over-smoothing when aggregating messages over long scheduling horizons.

Regarding the resource-constrained scenario presented in Fig. 7(c), the scarcity of AGVs and machines degrades the complex scheduling problem into a simpler priority queuing problem, limiting the solution space. Consequently, the convergence trends of MA-Trans, ResGAT, and LSTM-Ptr become tightly aligned. Nevertheless, MA-Trans still maintains a slight edge in stability and final convergence value, reaffirming its robustness even under highly constrained conditions.

Beyond average performance and convergence speed, stability remains a crucial metric for industrial applications. Fig. 8 presents the boxplots of the Relative Percentage Deviation (RPD) for all compared methods. Calculated relative to the best known solution found among the five algorithms for each instance, the RPD provides a normalized measure of robustness where a value closer to zero indicates superior consistency.

As illustrated in the plots, the three end-to-end DRL-based methods, namely MA-Trans, ResGAT, and LSTM-Ptr, generally outperform the hybrid and evolutionary baselines. Specifically, SA-PPO exhibits significant variance characterized by taller boxes and extended whiskers. This instability arises because the single-agent PPO lacks a collaborative mechanism, leading to frequent decision conflicts between the learned policy and the fixed dispatching rules, which prevents the agent from consistently finding high-quality solutions. Similarly, while MA performs adequately on small scales, its variance increases dramatically in large-scale scenarios as shown in Fig. 8(d). This deterioration reconfirms that the stochastic search process struggles to maintain population diversity and convergence precision in high-dimensional spaces.

In contrast, MA-Trans demonstrates superior robustness, characterized by the most compact interquartile range and a median RPD consistently approaching 0% across all scenarios. A distinct pattern emerges in Fig. 8(c), where the median RPD values of MA-Trans, ResGAT, and LSTM-Ptr are tightly aligned. This visual evidence aligns with our previous analysis regarding the resource-constrained bottleneck, where

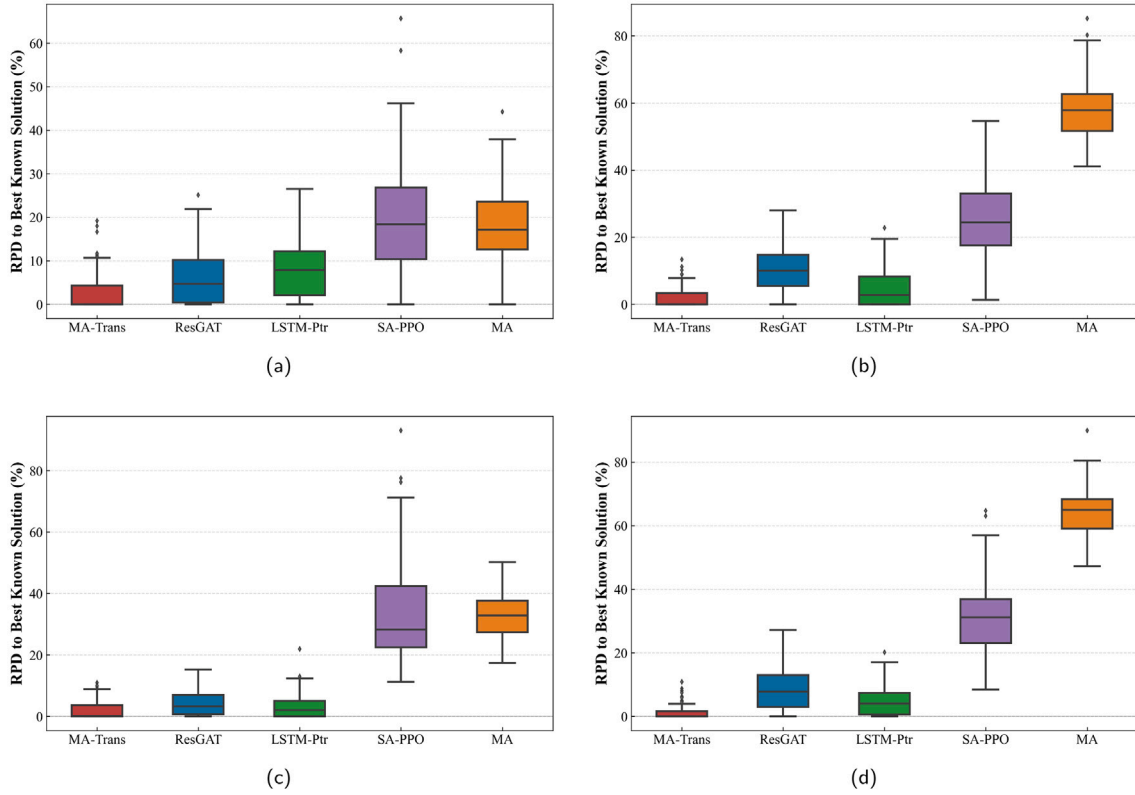


Fig. 8. Comparative analysis of solution quality distribution and stability via Relative Percentage Deviation (RPD) across four distinct scenarios – (a) Small-Scale Validation ($10 \times 5 \times 2$), (b) Medium-Scale Balanced ($15 \times 10 \times 4$), (c) Resource-Constrained Bottleneck ($20 \times 5 \times 3$), and (d) Large-Scale High-Density ($20 \times 10 \times 5$). The RPD is calculated as $(C_i - C_{\text{best}})/C_{\text{best}} \times 100\%$, where C_{best} denotes the best known solution among all five methods.

scarce system resources flatten the optimization landscape and reduce the margin for improvement. However, even in this highly constrained context, MA-Trans maintains a lower distribution of outliers compared to LSTM-Ptr. This further verifies that the global attention mechanism offers a more reliable policy execution than the recurrent architecture, effectively mitigating the risk of performance degradation in worst-case scenarios. Crucially, the performance gap between MA-Trans and SA-PPO serves as a decisive structural ablation. Although both methods utilize the same Attention mechanism for feature extraction, SA-PPO fails to achieve comparable stability due to its single-agent formulation. This strongly argues that the Dual-Agent Hierarchical Structure – which effectively decouples the exponential action space—is the primary driver for solving high-dimensional scheduling problems, rather than the representational power of Attention alone.

5.3. Zero-shot generalization capability

To verify the practical scalability and robustness of the proposed framework, we evaluate the zero-shot generalization capability of the policy trained on the $20 \times 10 \times 5$ dataset. The agent is directly applied to unseen scenarios without any fine-tuning. We compare MA-Trans against the best-performing heuristic, MWKR-EAT, to establish a baseline for industrial viability, and against two representative deep learning methods, ResGAT and LSTM-Ptr, to assess architectural superiority. The test instances are categorized into three distinct groups: downward generalization scenarios including 10×5 and 15×10 scales to verify efficiency on simpler tasks; a resource-constrained bottleneck scenario where the number of AGVs is reduced by 40% to test adaptability to severe congestion; and upward scalability scenarios such as 25×10 and 30×10 to challenge the model with larger state spaces, thereby testing the extrapolation capability of the attention mechanism.

As visualized in Fig. 9 and detailed in Table 6, MA-Trans consistently demonstrates superior generalization, achieving substantial performance gains over the MWKR-EAT rule across all scenarios, with improvements ranging from 7.7% to 32.0%. This universal advantage confirms that the agent has internalized a generalized scheduling logic that transcends specific instance sizes. Regarding downward generalization, the results highlight the architectural characteristics of different models. In small-scale instances like 10×5 , the performance gap between MA-Trans and LSTM-Ptr is negligible, with a p-value exceeding 0.05. This statistical similarity implies that when the solution space is small and the sequence length is short, the problem complexity is within the effective capacity of sequential models like LSTM. However, as the problem complexity increases to medium scales such as 15×10 , the global modeling advantage of the Transformer begins to manifest, leading to statistically significant improvements over the baselines.

In the resource-constrained bottleneck scenario of $20 \times 10 \times 3$, a notable saturation effect is observed. While all deep learning methods achieve a high improvement over the rule, the relative differences between them narrow significantly. This phenomenon suggests that when AGV resources become the absolute bottleneck, the scheduling problem degrades into a rigid queuing system. The physical constraints of the environment limit the potential for optimization, causing all learning-based methods to converge toward a similar asymptotic limit. Nevertheless, MA-Trans still achieves the lowest numerical makespan, proving its robustness even under high-congestion conditions.

Extending the evaluation to upward scalability scenarios, MA-Trans exhibits superior extrapolation capabilities compared to the baselines. In the 30×10 limit test, while the performance gains of ResGAT and LSTM-Ptr begin to plateau or decline, MA-Trans maintains a dominant lead, outperforming the rule by 25.9%. This confirms that unlike RNNs which suffer from information forgetting in long sequences, or GNNs which

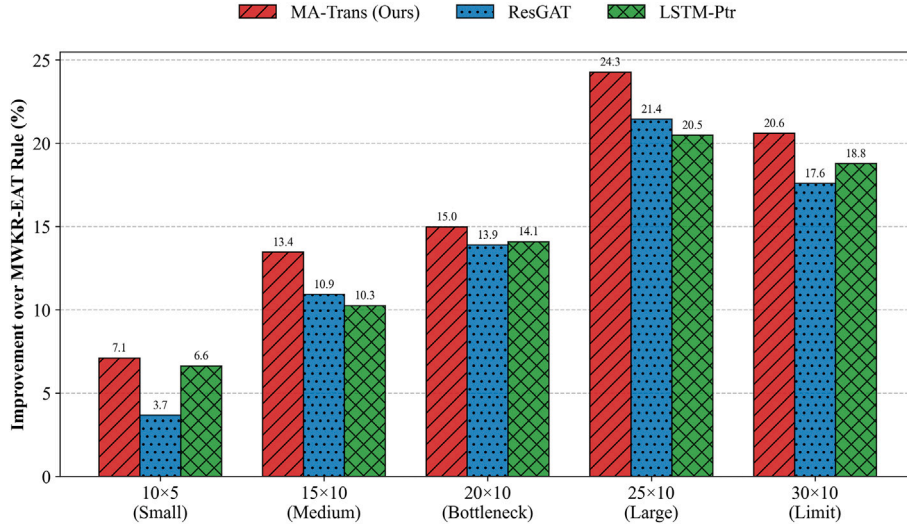


Fig. 9. Visualization of zero-shot generalization performance across five unseen scenarios. *Note:* The Y-axis denotes the improvement rate over the MWKR-EAT rule, defined as $(\text{Makespan}_{\text{Rule}} - \text{Makespan}_{\text{Method}}) / \text{Makespan}_{\text{Rule}}$. This metric highlights the advantage over heuristics, distinct from Table 6 where MA-Trans serves as the comparative baseline.

Table 6

Zero-shot generalization performance on unseen scenarios ($N = 100$).

Type	Scale	Method	Mean $\pm$ Std	Gap (%)	p-value
Downward (Small)	$10 \times 5 \times 2$	MWKR-EAT	253.6 ± 25.3	+7.7	< 0.001
		ResGAT	244.3 ± 23.4	+3.7	< 0.001
		LSTM-Ptr	236.8 ± 24.6	+0.5	0.403
		MA-Trans	235.6 ± 22.8	–	–
Downward (Medium)	$15 \times 10 \times 3$	MWKR-EAT	419.5 ± 32.8	+15.5	< 0.001
		ResGAT	373.7 ± 24.7	+2.9	< 0.001
		LSTM-Ptr	376.5 ± 26.3	+3.7	< 0.001
		MA-Trans	363.1 ± 25.5	–	–
Same (Bottleneck)	$20 \times 10 \times 3$	MWKR-EAT	551.1 ± 34.9	+17.6	< 0.001
		ResGAT	474.6 ± 30.3	+1.8	0.186
		LSTM-Ptr	473.5 ± 37.1	+1.5	0.203
		MA-Trans	468.6 ± 48.0	–	–
Upward (Large)	$25 \times 10 \times 5$	MWKR-EAT	507.1 ± 27.4	+32.0	< 0.001
		ResGAT	398.4 ± 22.5	+3.7	< 0.001
		LSTM-Ptr	403.3 ± 21.2	+5.0	< 0.001
		MA-Trans	384.1 ± 21.2	–	–
Upward (Limit)	$30 \times 10 \times 5$	MWKR-EAT	534.7 ± 22.5	+25.9	< 0.001
		ResGAT	440.7 ± 23.4	+3.8	< 0.001
		LSTM-Ptr	434.3 ± 22.9	+2.3	< 0.001
		MA-Trans	424.6 ± 23.8	–	–

Note: 1. **Model Source:** The policy trained on $20 \times 10 \times 5$ is applied directly without fine-tuning. 2. **MWKR-EAT:** The benchmark rule used as the baseline for Gap calculation. 3. **Gap:** Calculated as $(\text{Mean}_{\text{Method}} - \text{Mean}_{\text{Rule}}) / \text{Mean}_{\text{Rule}} \times 100\%$. Negative values indicate performance improvement over the rule. 4. **p-value:** Calculated via paired t -test against MA-Trans.

face over-smoothing in large graphs, the global attention mechanism of MA-Trans effectively retains long-range dependencies. This allows the policy to make globally coordinated decisions even when the problem scale expands by 50% relative to the training set.

5.4. Computational efficiency analysis

To provide a comprehensive evaluation of the trade-off between decision quality and computational cost, we performed a detailed efficiency analysis based on the $20 \times 10 \times 5$ problem scale. Table 7 summarizes the model complexity in terms of parameters and FLOPs alongside the training overhead, while Fig. 10 illustrates the average inference latency per operation.

Table 7

Comparison of model complexity and computational efficiency (20×10 Scale).

Method	Params (M)	FLOPs (M)	Training Time (h)	Inference (ms)	Train GPU Util. (%)	Inf. GPU Util. (%)
MA-Trans (Ours)	0.90	25.20	30.32	2.75	81.4	20.3
ResGAT	0.11	2.11	23.62	2.24	45.0	18.5
LSTM-Ptr	2.16	102.15	39.79	4.10	88.4	65.7

Note: Parameters (Params) and FLOPs are in Millions (M). GPU utilization is measured on an RTX 4060. Bold indicates the best result.

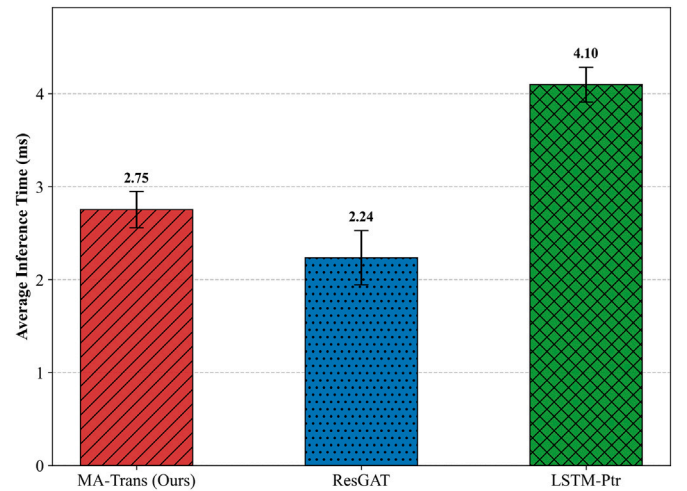


Fig. 10. Comparison of average inference latency per operation among MA-Trans, ResGAT, and LSTM-Ptr.

As indicated by the quantitative results in Table 7, the proposed MA-Trans architecture challenges the common assumption that Transformers are inherently computationally heavy. Compared to the LSTM-Ptr baseline, which incurs a substantial burden of 2.16 M parameters and 102.15 M FLOPs, our method significantly optimizes

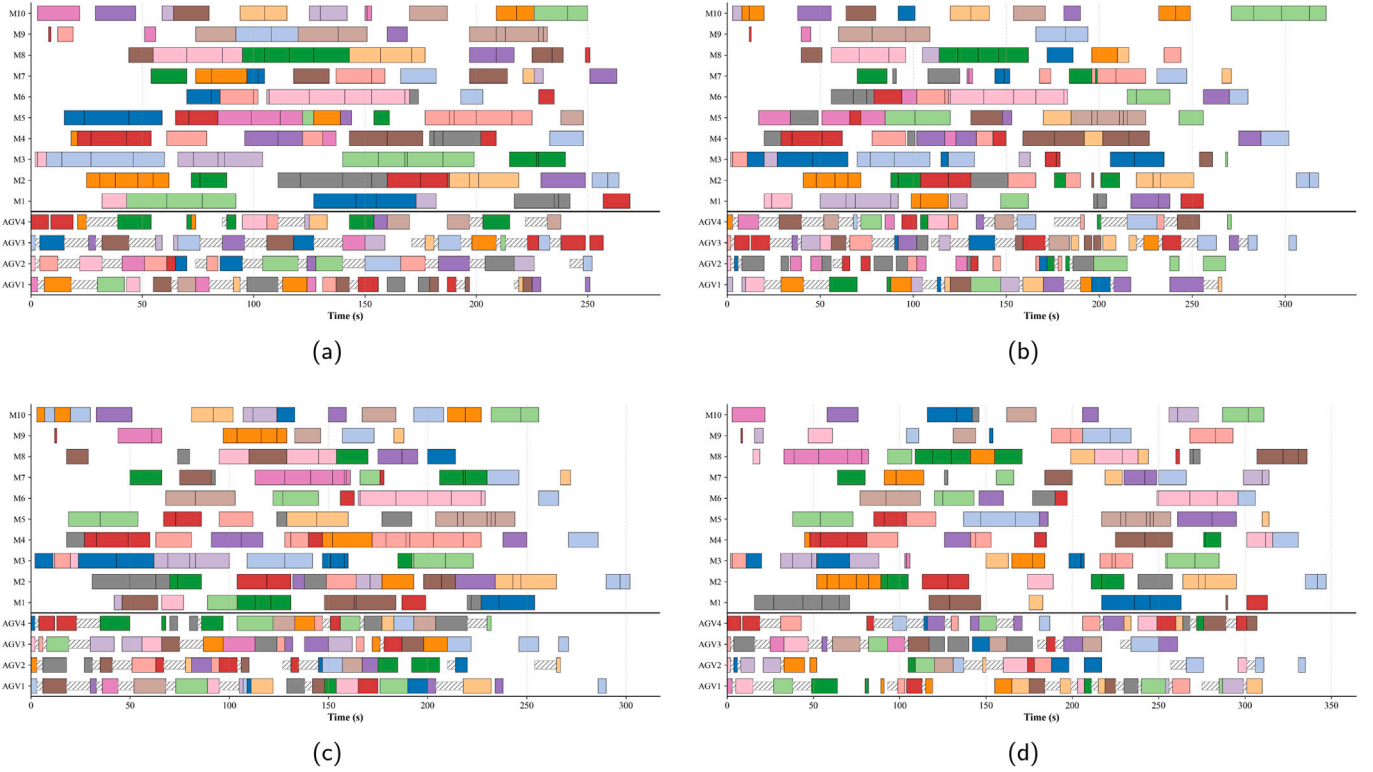


Fig. 11. Visual comparison of scheduling Gantt charts generated by different algorithms on a $15 \times 10 \times 4$ problem instance. (a) MA-Trans (Ours), Makespan: 267; (b) ResGAT, Makespan: 322; (c) LSTM-Ptr, Makespan: 302; (d) SA-PPO, Makespan: 348.

resource usage. By leveraging a non-autoregressive encoder, MA-Trans reduces the model size to 0.90 M parameters and cuts the floating-point operations to 25.20 M. Crucially, the GPU utilization metrics reveal distinct computational behaviors between the architectures. During the inference phase, MA-Trans maintains a remarkably low GPU utilization of 20.3%, in stark contrast to the heavy load of 65.7% exhibited by the LSTM baseline. This disparity highlights the efficiency of our non-autoregressive encoder: while LSTM saturates the GPU with sequential gate computations, our model executes decisions rapidly (2.75 ms) with minimal resource occupation, leaving ample headroom for real-time responsiveness. Furthermore, in the training phase, MA-Trans achieves a high utilization of 81.4%, significantly outperforming the graph-based ResGAT (45.0%). This indicates that the dense matrix multiplications in the Transformer architecture are highly compatible with modern hardware parallelization, effectively circumventing the sparse computation inefficiencies of GNNs and the sequential Backpropagation Through Time bottleneck of RNNs. Consequently, this structural efficiency translates into a faster convergence time of 30.32 hours, reducing the computational time by 23.8% compared to the 39.79 hours required by the LSTM baseline.

Furthermore, this efficiency advantage is preserved during the decision-making process, which is critical for online scheduling. As illustrated in Fig. 10, MA-Trans attains an average inference latency of 2.75 ms, significantly outperforming the 4.10 ms latency of LSTM-Ptr. This confirms that our dual-agent architecture effectively utilizes GPU parallelization, ensuring rapid response times suitable for dynamic industrial environments.

At the other end of the complexity spectrum, while the ResGAT baseline exhibits the lowest computational overhead with 0.11 M parameters and a 2.24 ms latency, this extreme simplification comes at the expense of representational capacity. As demonstrated in Section 5.3, ResGAT fails to generalize effectively to unseen scenarios due to its limited parameter space. Consequently, MA-Trans strikes an optimal

balance among the three methods. It invests a moderate amount of computational resources to capture complex global dependencies-achieving state-of-the-art performance significantly superior to ResGAT – while remaining far more lightweight and responsive than the traditional LSTM-based approach.

5.5. Case study and visualization

To provide an intuitive understanding of the scheduling quality and interpretable decision-making process, we selected a representative instance from the $15 \times 10 \times 4$ for detailed visualization. This medium-scale scenario strikes a balance between problem complexity and visual clarity, facilitating a clear examination of the coordination patterns between machines and AGVs without the visual clutter typical of larger instances. The Gantt charts generated by the proposed MA-Trans and three baseline methods are presented in Fig. 11.

As illustrated in Fig. 11(a), the schedule generated by MA-Trans exhibits a significantly more compact arrangement compared to the baselines. The operations on machines are tightly stacked with minimal idle gaps, indicating a high utilization rate of manufacturing resources. Beyond resource utilization, a critical advantage lies in the load balancing of AGVs. In the MA-Trans solution, transportation tasks are evenly distributed among the four AGVs, ensuring that each unit maintains a consistent and active workflow. This balanced workload prevents specific AGVs from becoming congestion bottlenecks while others remain idle, a common issue observed in the baseline schedules such as the uneven gaps shown in Fig. 11d. Consequently, this tight coordination and efficient resource allocation enable MA-Trans to achieve the shortest makespan of 267 s, significantly outperforming ResGAT at 322 s, LSTM-Ptr at 302 s, and SA-PPO at 348 s.

The superiority of the MA-Trans schedule can be fundamentally attributed to the global receptive field of the Transformer architecture and the cooperative dual-agent mechanism. Unlike the baselines that

often make myopic decisions based on local information, MA-Trans effectively captures the long-term spatio-temporal dependencies between job completion times and AGV availability. By implicitly anticipating future system states, the model proactively dispatches the most suitable free AGV to transport materials just-in-time, ensuring that machines are fed continuously without unnecessary waiting. This ability to globally synchronize heterogeneous processing and transportation resources is the key driver behind the high compactness, balanced AGV workload, and superior makespan observed in the proposed method.

6. Conclusion and future work

This paper proposes MA-Trans, a hierarchical deep reinforcement learning framework that integrates a dual-agent PPO architecture with Transformer encoders to solve the FJSP-AGV. To address the coupling complexity between operation sequencing and material handling, we decompose the problem into cooperative job scheduling and AGV dispatching sub-tasks. By leveraging the global receptive field of the self-attention mechanism, the model effectively captures long-range spatio-temporal dependencies among jobs, machines, and AGVs.

Regarding scheduling performance, comparative experiments demonstrate that MA-Trans significantly outperforms state-of-the-art baselines, including ResGAT and LSTM-Ptr. This superiority is attributed to the model's ability to implicitly coordinate machine availability with AGV transportation, resulting in the high compactness and load balancing observed in the Gantt chart visualizations. In terms of computational efficiency, unlike recurrent architectures that suffer from sequential bottlenecks, the parallelizable nature of our Transformer-based encoder reduces training time by approximately 23.8% compared to LSTM-based methods while maintaining a lightweight parameter footprint (0.90 M). Furthermore, the model exhibits strong zero-shot generalization. The policy trained on the standard 20×10 scale maintains performance stability when transferred to unseen instances without retraining, validating that the Transformer encoder successfully learns the underlying scheduling logic.

Despite these promising results, several limitations remain to be addressed in future work. One critical direction is to enhance the environmental realism. Since the current framework assumes ideal pathfinding, future studies will integrate complex kinematic constraints and Multi-Agent Path Finding (MAPF) algorithms to explicitly handle dynamic collisions and deadlocks in dense AGV environments. Furthermore, to address the multi-faceted nature of manufacturing, we plan to extend the framework to multi-objective optimization, utilizing Pareto-based rewards to balance makespan with conflicting goals such as energy consumption and tardiness. It is also worth exploring hybrid architectures to better capture structural dependencies. Acknowledging the inherent graph topology of shop floors, future research could combine Graph Neural Networks (GNNs) for local topology encoding with Transformers for global temporal reasoning, aiming to further improve representation learning.

CRedit authorship contribution statement

Yuxiang Shen: Writing – review & editing, Writing – original draft, Validation, Methodology. **Xutang Zhang:** Conceptualization. **Tianguo Jin:** Conceptualization.

Funding

This work was supported by the **National Key Research and Development Program of China** (Grant No. 2023YFB4302003).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] B. Acherjee, Fem-Ann sequential modelling of laser transmission welding for prediction of weld pool dimensions, in: *Non-Conventional Machining in Modern Manufacturing Systems*, IGI Global Scientific Publishing, 2019, pp. 249–261.
- [2] B. Acherjee, S. Mondal, B. Tudu, D. Misra, Application of artificial neural network for predicting weld quality in laser transmission welding of thermoplastics, *Appl. Soft Comput.* 11 (2011) 2548–2555.
- [3] M.K. Amjad, S.I. Butt, N. Anjum, I.A. Chaudhry, Z. Faping, M. Khan, A layered genetic algorithm with iterative diversification for optimization of flexible job shop scheduling problems, *Adv. Prod. Eng. Manag.* 15 (2020) 377–389, <https://doi.org/10.14743/APEM2020.4.372>
- [4] G. Anwer, B. Acherjee, A deep learning-based data-driven approach for modeling and optimization of laser transmission welding of polypropylene, *Int. J. Ind. Eng. Theory Appl. Pract.* 32 (2025).
- [5] O.T. Baruwaa, M.A. Piera, A coloured petri net-based hybrid heuristic search approach to simultaneous scheduling of machines and automated guided vehicles, *Int. J. Prod. Res.* 54 (2017) 1–20, <https://doi.org/10.1080/00207543.2016.1154203>
- [6] P. Brucker, B. Jurisch, M. Sievers, Shop scheduling problems with transportation times, *Ann. Oper. Res.* 50 (1994) 433–451.
- [7] J. Chen, X. Zhu, C. Chakraborty, M. Guduri, A. Alharbi, A. Tolba, K. Yu, Dynamic optimization of vehicle production planning in transportation networks using federated reinforcement learning, *IEEE Trans. Intell. Transp. Syst.* (2025).
- [8] B. Cunha, A. Madureira, B. Fonseca, et al., Deep reinforcement learning as a job shop scheduling solver: a literature review, in: *Proceedings of the International Conference on Hybrid Intelligent Systems*, 2018, pp. 350–359.
- [9] Z. Dong, T. Ren, F. Qi, J. Weng, D. Bai, J. Yang, C.-C. Wu, A reinforcement learning-based approach for solving multi-agent job shop scheduling problem, *Int. J. Prod. Res.* 63 (2025) 3512–3537.
- [10] A. Duggirala, B. Acherjee, S. Mitra, Predicting weld pool metrics in laser welding of aluminum alloys using data-driven surrogate modeling: a Fea-Doe-Gprn hybrid approach, *Proc. Inst. Mech. Eng. Part E J. Process Mech. Eng.* (2024) 09544089241255927.
- [11] D. Fontes, S.M. Homayouni, Joint production and transportation scheduling in flexible manufacturing systems, *J. Glob. Optim.* 74 (2019) 879–908, <https://doi.org/10.1007/s10898-018-0706-6>
- [12] X. Jing, Research on Flexible Job Shop Scheduling Problem Based on Multi-Agent Deep Reinforcement Learning. (Ph.D. thesis). South China University of Technology, 2023. In Chinese.
- [13] M. Khadivi, et al., Deep reinforcement learning for machine scheduling: Methodology, the state-of-the-art, and future directions, *arXiv preprint arXiv:2310.03195*, 2023.
- [14] D. Kumar, S. Ganguly, B. Acherjee, A.S. Kuar, Performance evaluation of twist welding using machine learning assisted evolutionary algorithms, *Arab. J. Sci. Eng.* 49 (2024) 2411–2441.
- [15] F. Li, et al., A transformer-based deep reinforcement learning approach for dynamic parallel machine scheduling problem with family setups, *J. Intell. Manuf.* (2024), <https://doi.org/10.1007/s10845-024-02470-8>. online first.
- [16] Y. Liao, et al., A distributed scheduling method for intelligent workshops based on multi-agent reinforcement learning, in: *Proceedings of the 2024 China Automation Congress, School of Electronics and Information Engineering, Tongji University; Shanghai Research Center for Autonomous Intelligent Unmanned Systems*, Shanghai, China, 2024, pp. 515–520. In Chinese.
- [17] C.L. Liu, C.C. Chang, C.J. Tseng, Actor-critic deep reinforcement learning for solving job shop scheduling problems, *IEEE Access* 8 (2020) 71752–71762, <https://doi.org/10.1109/ACCESS.2020.2987680>
- [18] G. Liu, M. Liao, W. Chen, Z. Zhang, H. Wan, Y. Lin, Multi-faceted dynamic flexible job shop scheduling via heterogeneous graph transformer and deep reinforcement learning, *Expert Syst. Appl.* (2025) 130532.
- [19] Q. Liu, et al., Research on flexible job shop scheduling optimization based on segmented AGV, *C.M.E.S. - Comput. Model. Eng. Sci.* 134 (2022) 2073–2091, <https://doi.org/10.32604/cmes.2022.021433>, <https://www.sciencedirect.com/science/article/pii/S152614922200073X>.
- [20] X. Liu, Y. Wang, R. Zhao, A multi-agent deep reinforcement learning approach for dynamic flexible job shop scheduling with transportation constraints, *Appl. Soft Comput.* 113 (2021) 107899, <https://doi.org/10.1016/j.asoc.2021.107899>
- [21] S. Luo, Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning, *Appl. Soft Comput.* 91 (2020) 106208, <https://doi.org/10.1016/j.asoc.2020.106208>
- [22] F. Meng, et al., Solving the integrated scheduling problem in flexible job shops based on multi-agent reinforcement learning, *Comput. Integr. Manuf. Syst.* (2024) 1–29, <https://doi.org/10.13196/j.cims.2024.0236>. in Chinese. Forthcoming or online first.
- [23] H. Nabovati, H. Haleh, B. Vahdani, Multi-objective invasive weeds optimisation algorithm for solving simultaneous scheduling of machines and multi-mode automated guided vehicles, *Eur. J. Ind. Eng.* 14 (2020) 165–188,

- <https://doi.org/10.1504/EJIE.2020.105696>, <https://www.inderscience.com/info/inarticle.php?artid=105696>.
- [24] Z. Shi, Q. Wang, J. Zhang, C. Luo, H. Chen, G. Ding, A hybrid nsga-ii for energy-efficient flexible job shop scheduling problem with finite multi-load AGV resources, *J. Eng. Des.* (2025) 1–30.
 - [25] W. Song, X. Chen, Q. Li, Z. Cao, Flexible job shop scheduling via graph neural network and deep reinforcement learning, *IEEE Trans. Ind. Informat.* 19 (2023) 1600–1610, <https://doi.org/10.1109/TII.2022.3189725>
 - [26] S. Tian, C. Huangfu, X. Deng, A review of application scenarios, key factors, and research methods in AGV scheduling optimization, *J. Chongqing Univ. Posts Telecommun. (Nat. Sci. Ed.)* 36 (2024) 337–350. In Chinese.
 - [27] Y. Tian, Y. Tian, X. Liu, A review of algorithms for solving flexible job shop scheduling problems, *J. Yan'an Univ. (Nat. Sci. Ed.)* 40 (2021) 64–70, <https://doi.org/10.13876/J.cnki.ydnse.2021.03.064>. In Chinese.
 - [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008, https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
 - [29] C. Wang, Q.-K. Pan, A multi-objective two-phase cascaded memetic algorithm for multi-shop integrated scheduling problem with AGV transportation, *Memet. Comput.* 17 (2025) 29.
 - [30] J. Wang, J. He, J. Zhang, A reinforcement learning method to optimize the priority of product for scheduling the large-scale complex manufacturing systems, in: *Proceedings of the 48th International Conference on Computers & Industrial Engineering (CIE48)*, 2018, pp. 2–5.
 - [31] Q. Wang, P. Guo, Z. Ma, X. Meng, A multi-agent deep reinforcement learning for flexible job shop scheduling problem with transportation, in: *Design and Operation of Smart Reconfigurable Manufacturing Systems in Industry 4.0/5.0*, Springer, 2025, pp. 123–142.
 - [32] Y. Wang, J. Zhang, Y. Liu, A review of research on flexible job shop scheduling problems, *Comput. Integr. Manuf. Syst.* 25 (2019) 1102–1112. In Chinese.
 - [33] Z. Xie, Y. Xia, A scheduling method for idle capacity allocation across multiple workshops based on genetic algorithm and branch-and-bound, *J. Mech. Eng.* 58 (2022) 462–472. In Chinese.
 - [34] C. Zhang, W. Song, Z. Cao, et al., Learning to dispatch for job shop scheduling via deep reinforcement learning, in: *Advances in Neural Information Processing Systems*, 2020, pp. 1621–1632, <https://proceedings.neurips.cc/paper/2020/file/7a677bb44af3c5be74203df3c5cfa0b8-Paper.pdf>.
 - [35] W. Zhang, F. Zhao, B. Feng, X. Mei, A reinforcement learning control framework based on scalable graph transformer for large-scale fuzzy job shop scheduling problems, *IEEE Trans. Neural Netw. Learn. Syst.* (2025).
 - [36] W. Zhen, J. Chen, Research on influencing factors of production capacity in manufacturing workshops, *Light Ind. Mach.* 38 (107) (2020) 99–102. In Chinese.
 - [37] Y. Zhou, et al., A dqn-based scheduling method for discrete manufacturing workshops considering AGV transportation, *J. Mech. Eng.* 60 (2024) 338–348. In Chinese.
 - [38] C. Zhu, X. Li, Y. Yang, Research progress on intelligent manufacturing scheduling based on reinforcement learning, *Manuf. Autom.* 45 (2023) 1–6. In Chinese.
 - [39] Z. Zhu, W. Zhang, H. Ma, A collision-free AGV scheduling method for flexible job shop using graph neural network and reinforcement learning, *Robot. Comput.-Integr. Manuf.* 70 (2021) 102135, <https://doi.org/10.1016/j.rcim.2021.102135>